Department of Electrical and Computer Engineering
North South University

Senior Design Project

Privacy-Preserving Hybrid FHE Autoregressive LLM Inference via Selective Linear Offloading and Cascaded Calibration

Md Imam Hossain

ID: 2011756042

Faculty Advisor:

Dr. Nabeel Mohammed

Associate Professor

ECE Department

Spring 2025

LETTER OF TRANSMITTAL

April, 2025

To

Dr. Mohammad Abdul Matin
Chairman,
Department of Electrical and Computer Engineering
North South University, Dhaka

Subject: **Submission of Capstone Project Report on "Privacy-Preserving Hybrid FHE Autoregressive LLM Inference via Selective Linear Offloading and Cascaded Calibration"**

Dear Sir,

With due respect, I am submitting my Capstone Project Report entitled "Privacy-Preserving Hybrid FHE Autoregressive LLM Inference via Selective Linear Offloading and Cascaded Calibration" in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering. The work was carried out under the supervision of Dr. Nabeel Mohammed.

This project investigates privacy-preserving autoregressive large language model inference through a hybrid fully homomorphic encryption architecture. The report presents the system design, the compilation and cascaded calibration workflow, and the experimental evaluation of quality, latency, and communication trade-offs.

I have tried to complete the work sincerely and to present the report in a clear and technically faithful manner. I hope it will meet the academic requirements of the program.

Sincerely yours,

Md Imam Hossain
ID: 2011756042

Approval

Md Imam Hossain (2011756042), a student of Bachelor of Science in Computer Science and Engineering at North South University, has carried out the Senior Design Project titled **Privacy-Preserving Hybrid FHE Autoregressive LLM Inference via Selective Linear Offloading and Cascaded Calibration** under the supervision of **Dr. Nabeel Mohammed** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering. The work is accepted as satisfactory in scope and quality.

Supervisor's Signature

Dr. Nabeel Mohammed
Associate Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Chairman's Signature

Dr. Mohammad Abdul Matin
Professor and Chair
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Declaration

This is to declare that this project is my original work. No part of this work has been submitted elsewhere, partially or fully, for the award of any other degree or diploma. Relevant previous works presented in this report have been properly acknowledged and cited. To the best of my knowledge, the results, implementation details, and discussions presented here faithfully reflect the work carried out during this capstone project.

Student's Name and Signature

Md Imam Hossain

Acknowledgements

I am deeply grateful to my supervisor, **Dr. Nabeel Mohammed**, Associate Professor, Department of Electrical and Computer Engineering, North South University, for his guidance throughout this project. His feedback repeatedly forced me to sharpen the problem statement, justify design decisions more carefully, and treat implementation evidence with the level of discipline that this topic demands.

I would also like to thank Department of Electrical and Computer Engineering, North South University, for providing the academic setting in which this work was carried out. Finally, I remain grateful to my family for their patience and support during the long periods of debugging, compilation, experimentation, and writing that shaped the final outcome.

Abstract

Large Language Models are typically served from cloud infrastructure, which requires users to reveal prompts to the service operator. This work investigates a hybrid alternative in which only the transformer's linear projections are executed under Fully Homomorphic Encryption on the server, while the non-linear path remains on a trusted client.

The resulting system is a full 22-layer autoregressive TinyLlama 1.1B runtime. The server evaluates the `q`, `k`, `v`, and `o` projections of every layer under FHE, while the client performs embedding lookup, rotary position handling, attention computation, KV-cache maintenance, normalization, multilayer perceptron execution, and token selection. A staged offline compiler supports this split by constructing separate prefill and decode calibration sets, propagating calibration state across layers through cascaded simulation, and packaging 176 deployable circuits together with reference-free client and server artifacts.

In the full 22-layer `8/8/8` configuration, the system achieved average semantic similarity of 0.4852, token accuracy of 0.4833, BLEU of 0.3451, and an FHE-to-plaintext perplexity ratio of 1.3611x across ten prompts under a maximum of five new tokens per prompt. Average end-to-end latency was 3396.285 s, and returned ciphertext volume exceeded 19 GB per prompt. Additional experiments show that cascaded calibration improves retained quality at depth, whereas communication remains the dominant practical bottleneck.

Taken together, the work contributes a full-depth hybrid autoregressive runtime, a cascaded-calibration compiler workflow, and an end-to-end measurement study showing how compiler design, encrypted depth, and ciphertext movement shape the feasibility of privacy-preserving LLM serving.

Keywords: fully homomorphic encryption, privacy-preserving inference, large language model, TinyLlama, Concrete-ML, hybrid client-server system

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Large Language Models (LLMs) have moved quickly from research prototypes to everyday tools for drafting, summarization, coding assistance, and interactive analysis. What has not changed at the same pace is the trust model under which most of these systems are served. In the common cloud setting, the user sends a prompt to remote infrastructure, the provider executes the model in plaintext, and the response is sent back over the network. The operational advantages are obvious, but so is the cost: the provider gains visibility into the prompt itself.

That visibility is not a minor implementation detail. In many realistic workflows, the prompt contains the most sensitive part of the interaction: internal policy language, personal records, legal facts, confidential business logic, or draft analysis that an institution may not want to disclose to an external service operator. Transport security helps while data is in transit, but it does nothing once the server begins inference. In other words, if cloud inference is the deployment model, prompt disclosure is built into the architecture rather than added by mistake.

Fully Homomorphic Encryption (FHE) is attractive precisely because it challenges that assumption. It promises a way to compute on encrypted data without first recovering the plaintext [1], [2], [3]. If that promise could be brought to modern language-model inference at acceptable cost, a provider could assist with computation without directly reading what the user submitted. The difficulty is that transformer inference is not a simple numerical pipeline. It mixes large linear projections with normalization, masking, stateful decoding, attention updates, and token-by-token control flow. The challenge is therefore not only to run encrypted layers once, but to preserve a correct causal decode loop while the hidden state repeatedly crosses the client-server boundary.

The motivation for this work comes from that tension between what FHE offers in principle and what current systems can deliver in practice. Rather than claim a solution to fully encrypted transformer inference, this project takes a narrower but more durable question: **how much privacy and computational offloading can be recovered by pushing the transformer's linear**

bulk behind FHE while keeping the non-linear and stateful parts on the client, where current tooling still handles them more naturally, and can that split be sustained through a full autoregressive runtime? The point of the split is not to avoid strong servers or future accelerators. It is to avoid paying encrypted cost for the least FHE-friendly part of the transformer in the first place.

1.2 Problem Statement

The practical use of LLMs in sensitive settings is constrained by an uncomfortable trade-off. If inference is delegated to a provider, prompt confidentiality is weakened. If confidentiality must be preserved, the user is often pushed toward full local execution or toward much heavier secure-computation designs. Existing FHE literature establishes the cryptographic basis for computing on encrypted data and shows that selected neural workloads can be adapted successfully, but autoregressive language generation remains constrained by compute cost, non-linear transformer components, state management, calibration mismatch, key handling, and communication overhead [4, 5, 6, 7, 8, 9], [10].

The engineering problem addressed in this project is therefore the following:

> Design and evaluate a reproducible hybrid client-server framework in which the large linear portions of transformer inference execute under Fully Homomorphic Encryption on the server, while non-linear and decode-state operations execute locally on a trusted client because those operations remain comparatively costly, approximate, or awkward under the current FHE stack, and demonstrate that this split can sustain a real token-by-token autoregressive decode loop rather than only an isolated encrypted subroutine.

In that form, the problem is not only cryptographic but also architectural: the system must determine where to cut the model, how to keep autoregressive decoding coherent across that cut, and how to calibrate the encrypted subgraph so that quality does not collapse as encrypted depth increases.

1.3 Objectives

The project objectives are:

1. To design a hybrid execution strategy for TinyLlama 1.1B that encrypts only the transformer sub-operations that are practically deployable under the chosen FHE toolchain.
2. To build a fully autoregressive client-server runtime with prefill, token-by-token decode, local KV-cache management, and reference-free client execution.

3. To build an offline compiler and packaging workflow that compiles all 22 layers of the selected encrypted subgraph into deployable FHE circuits while exporting the remaining model blocks for reference-free local execution.
4. To develop a staged calibration workflow that separates prefill and decode data, uses real KV context for decode calibration, and reduces the mismatch between clean calibration data and deep hybrid encrypted execution.
5. To study how encrypted depth, bit-width, calibration, and generation length affect output quality, runtime, and communication cost.
6. To document the key engineering decisions, calibration strategy, limitations, and future work required to make such systems more practical.

1.4 Scope and Constraints

The scope of the project is intentionally bounded so that the work remains defensible as a capstone implementation and feasibility study.

1. The model target is **TinyLlama 1.1B**, not a larger instruction-tuned frontier model.
2. The implemented encrypted subgraph is limited to the attention projections `q`, `k`, `v`, and `o`; attention score computation, SoftMax, normalization, key-value cache management, and MLP blocks remain local.
3. The system is evaluated primarily on **short-form generation** over ten prompts, with five-token generation as the main benchmark and longer generation as a secondary stress case.
4. The reported experiments were collected in a **local evaluation environment**, which served as the validation platform for the study rather than the intended ceiling of the architecture.
5. The server is modeled as **honest-but-curious**; the client is trusted.
6. MLP-under-FHE was considered during project planning but not delivered in the final system because extending encrypted coverage beyond the attention projections required more compilation time, memory, calibration work, and encrypted runtime than could be validated carefully within the project.

1.5 Organization of the Report

Chapter 2 reviews the literature on transformer models, FHE schemes, and privacy-preserving neural inference, with particular attention to the gap addressed by the present system. Chapter 3 presents the methodology, architecture, threat model, compilation pipeline, and runtime design. Chapter 4 evaluates the main design levers introduced in Chapter 3, including encrypted depth, bit-width, calibration, error-budget choice, generation length, and communication behavior. Chapter 5 examines privacy, societal, legal, and sustainability impacts. Chapter 6 summarizes project planning and budget. Chapter 7 maps the work to Complex Engineering Problems

(CEP) and Complex Engineering Activities (CEA). Chapter 8 concludes the report with the main limitations and future directions.

Chapter 2

Research Literature Review

2.1 Transformer Models and the Privacy Problem

Transformers became the dominant architecture for language modeling once self-attention proved that long-range sequence modeling could be handled without the recurrence used in earlier neural language models [11]. Later open-weight families such as LLaMA expanded access to transformer research and deployment [12], while smaller derivatives such as TinyLlama demonstrated that meaningful language-model behavior could still be studied at a scale more accessible to academic experimentation [13]. That matters for this project because it shifts the question from model capability alone to model deployment under realistic constraints.

From a privacy standpoint, the deployment question is central. In the usual service model, a provider sees the prompt, processes it, and returns text. That workflow may be acceptable for casual interactions, but it becomes difficult to justify when prompts contain internal institutional knowledge, confidential records, or high-stakes drafting material. The more useful language models become, the more consequential that trust assumption becomes as well.

The literature on language models therefore points in two directions at once. One concerns model quality and scale. The other concerns the conditions under which that capability is delivered. The present work is concerned with the second direction: how to preserve more privacy when the model is served through a managed system.

2.2 Fully Homomorphic Encryption Foundations

The idea of computing over encrypted data predates modern machine learning by decades [1], but for a long time it remained more of a conceptual goal than a deployment option. Gentry's 2009 construction changed that by showing how bootstrapping could be used to refresh ciphertexts and sustain deeper computation [2]. Since then, the field has diversified into several families of schemes, each of which makes different trade-offs in arithmetic style, packing model, programmability, and performance.

Those trade-offs are not abstract in a machine-learning context. They influence what kind of numerical operations are natural, how efficiently vectors can be packed, and how painful it is to implement the non-linear parts of modern neural networks. For transformer inference, those choices become especially important because the workload mixes many large linear maps with a smaller number of operations that are mathematically or operationally awkward under FHE.

Table 2.1 summarizes the scheme families most relevant to this project.

Table 2.1: Comparison of representative FHE schemes

Scheme family	Arithmetic style	Typical strengths	Typical limitations	Relevance to this project
BFV/BGV	Exact integer arithmetic	Strong algebraic foundations, useful for exact integer workflows	Less convenient for approximate real-valued ML pipelines	Not used directly in this project
CKKS	Approximate arithmetic on real numbers	Efficient SIMD-style batched linear algebra, widely used in HE-for-ML literature [14]	Approximate semantics, non-linearities still difficult	Important comparative baseline in literature
TFHE	Gate- and table-oriented integer computation with programmable bootstrapping [3]	Flexible encrypted circuit evaluation, strong tooling in Concrete-ML [8]	High computational cost, large ciphertexts, expensive deep pipelines	Selected scheme/toolchain in this project

Representative exact-integer schemes in the BFV/BGV family include the BGV line of leveled homomorphic schemes [15] and the BFV variant developed in the ring-LWE setting [16].

For this project, the practical implication is straightforward: the FHE scheme is inseparable from the engineering design. It affects not only runtime cost, but also which parts of the model are reasonable candidates for encryption in the first place.

This capstone uses **Concrete-ML** because it offers a Python-oriented path from quantized model components to deployable client/server artifacts [8]. More importantly for the present architecture, the current Concrete documentation explicitly distinguishes between fast linear arithmetic and non-linear computation realized through table lookups or programmable bootstrapping, whose cost depends strongly on bit-width [17], [18]. Zama's own LLM guidance goes further and frames encrypted LLM inference as a hybrid client/server protocol in which the client keeps the non-linear layers while the server evaluates the linear layers under FHE [19]. The general linear/non-linear split is therefore already visible in current tooling. The harder question is how far that split can be carried in a real causal decoder, how calibration should remain aligned with runtime state, and what trade-offs appear once the system is measured end to end.

2.3 Prior Work in Privacy-Preserving Neural Inference

Early systems such as **CryptoNets** demonstrated that encrypted neural inference was possible at all, but they did so on comparatively small and carefully adapted workloads [4]. **CHET** pushed the systems conversation forward by treating homomorphic inference as a compilation problem rather than only a cryptographic one [5]. Both contributions remain important, yet neither addresses the particular difficulty of autoregressive language generation, where computation unfolds token by token and state must be preserved across the decode loop.

Transformers make the problem harder for structural reasons. Their performance depends on repeated linear projections, but also on normalization, masking, SoftMax, and stateful attention updates. Those are exactly the kinds of operations that become inconvenient, expensive, or numerically fragile under current encrypted execution models. In other words, the challenge is not uniformly distributed across the transformer: linears are the natural FHE target, while nonlinear stages remain the architectural pain point. **THE-X** is valuable here not only because it shows that encrypted transformer inference can be made to work, but because it shows how much approximation and architectural adaptation are usually required before that becomes tractable [6].

Later LLM-focused work reinforces the same point from a systems angle. **Mittal** emphasizes the gap between cryptographic feasibility and deployable privacy-preserving inference once realistic model sizes and client-server constraints are taken seriously [7]. Current Concrete-ML documentation already assumes that practical encrypted LLM inference will use a hybrid protocol rather than a fully encrypted end-to-end stack [19]. **EncryptedLLM** shows that the field has also progressed on the full-FHE side by accelerating GPT-2 inference with GPU-backed implementations [10]. **Power-Softmax** pushes further by redesigning attention-related computation itself so that more of the transformer becomes compatible with encrypted execution [9]. The lesson from this literature is twofold. First, encrypted transformer inference is no longer speculative. Second, broad claims such as ”hybrid encrypted LLM inference exists” are no longer very informative by themselves. The harder and more valuable question is how a deep causal runtime is actually made to work under that split, and how it is kept numerically stable as encrypted depth increases.

2.4 Research Gap

The relevant gap for this capstone is not the absence of prior FHE literature. It is the relative shortage of engineering-grounded studies that show what happens when privacy-preserving transformer inference is pushed under realistic implementation constraints. Five observations from the literature are especially important:

1. Most mature FHE-for-ML demonstrations still target smaller models, modified architectures, or more static workloads than repeated causal decode in an open-weight LLM.

2. Many approaches rely on architecture substitutions or approximation strategies that significantly modify the original transformer.
3. Practical deployment issues such as evaluation-key handling, session reuse, transport protocol design, communication volume, artifact packaging, and runtime observability are often underreported relative to raw cryptographic feasibility.
4. The interaction between quantization calibration and autoregressive generation quality remains a major practical variable, especially when calibration must stay aligned with KV-cache growth and decode-time state.
5. Although current tooling already exposes a hybrid client/server path, there remains relatively little public, engineering-grounded evidence on what happens when an open-weight decoder is pushed to full depth under that split, with separate prefill/decode treatment, compile-time calibration that is coupled to runtime state, and end-to-end communication analysis [17, 18, 19].

That is the gap addressed in this capstone. Existing tools and recent literature already support the broad idea of a hybrid linear-server / non-linear-client split. What remains less well documented is what happens when that split is carried through a full causal decoder, with calibration, packaging, session handling, and runtime state treated as one coupled system.

The study contributes in three specific ways. First, it develops a full 22-layer hybrid TinyLlama runtime in which encrypted attention projections are integrated into a real autoregressive prefill-and-decode loop, with explicit KV-cache handling, RoPE position tracking, and repeated client-server re-entry rather than an isolated proof of concept. Second, it implements and studies a compiler-side workflow that couples separate prefill/decode calibration, layer-aware cascaded simulation, and separate `o`-projection calibration from quantized attention outputs. Third, it records the quality, latency, calibration, security-boundary, and communication trade-offs that emerge from that design so that future work can target concrete bottlenecks rather than rely on general statements about privacy-preserving inference.

Chapter 3

Methodology

3.1 System Design Overview

The methodology is built around one practical question: where should the encryption boundary be placed so that privacy improves without making the system collapse under its own cost? The answer adopted in this project is to **push only the FHE-compatible linear subgraph to the server and keep the rest of the transformer local on a trusted client**. This yields a hybrid client-server framework rather than an end-to-end encrypted transformer. The client retains the secret key, performs all decryption, and handles the operations that remain impractical under the same FHE budget. The server is limited to the encrypted linear projections that can be compiled reliably under the chosen toolchain.

This design choice is architectural rather than anti-acceleration. Current FHE tooling already makes linear maps much more natural targets than SoftMax, normalization, activation functions, and other non-linear stages [17, 18, 19]. Even if future GPU, FPGA/HPU, or ASIC backends reduce encrypted latency substantially [20, 21, 22], excluding the least FHE-friendly part of the transformer would still reduce encrypted cost, energy use, and communication burden.

This split is implemented at every TinyLlama layer through the same repeated execution pattern:

1. The client performs embedding lookup and local normalization.
2. The normalized hidden state is encrypted and sent to the server for FHE execution of `q`, `k`, and `v`.
3. The client decrypts `q`, `k`, and `v`, applies rotary position embedding, updates the key-value cache, computes attention scores locally, and forms the attention context.
4. The attention context is encrypted and sent to the server for FHE execution of `o`.
5. The client decrypts the `o` projection and finishes the layer locally through residual and MLP/body execution.
6. Final normalization, language-model head projection, token selection, and decode-loop

control remain local.

What emerges from this split is a repeatable layer protocol rather than a single encrypted block inserted for demonstration. That protocol runs once during prefill and then repeats during token-by-token decode. The design makes no attempt to hide the fact that plaintext reappears on the client after each encrypted step. Its value lies elsewhere: the server can assist with computation without seeing the prompt, the hidden-state vectors it processes, or the generated continuation in plaintext. In that sense, the methodological contribution is not just an encrypted layer extraction. It is a complete causal runtime built around that extraction.

This distinction matters because a full-length completion at the five-token cap is not one encrypted pass through the model. It is one prefill pass plus four decode rounds, with the client preserving state across the split and re-entering the encrypted path at every step. At 22 layers, that requires 440 encrypted circuit calls. The central systems challenge is therefore not only compiling encrypted linears, but keeping a causal generation runtime coherent while those linears are repeatedly invoked inside the loop.

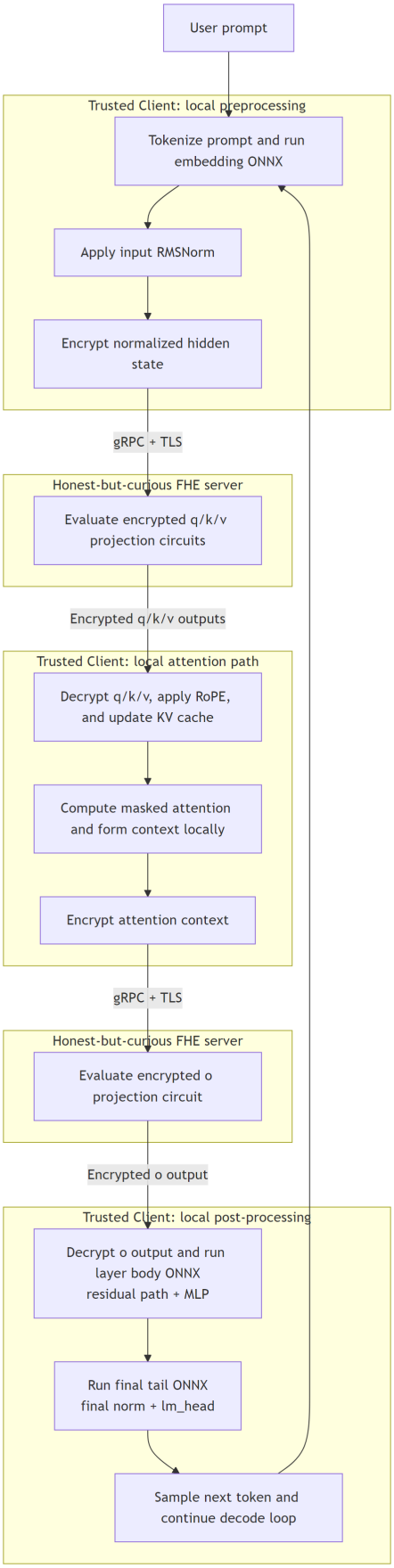

Figure 3.1: Runtime architecture of the hybrid FHE system

3.2 Threat Model and Security Boundary

The threat model has one trusted party and one partially trusted party. The client is trusted: it generates and retains the secret key, encrypts inputs, decrypts returned ciphertexts, and manages all plaintext post-processing. The server is **honest-but-curious**: it is assumed to execute the protocol correctly, but it may attempt to infer information from whatever it can observe.

Under that model, the primary security objective is narrow and explicit: **the server should not learn the plaintext prompt, plaintext hidden states, or generated tokens from the inference workflow itself**.

Protected data in the current design includes:

1. Prompt tokens and prompt-derived hidden states.
2. Encrypted inputs to server-side `q`, `k`, `v`, and `o` circuits.
3. Returned encrypted outputs before local decryption.
4. The client's secret key material.

Data not protected from a trusted-client compromise includes:

1. Locally decrypted `q`, `k`, `v`, and `o` outputs.
2. Attention scores, KV-cache state, and MLP activations.
3. Final generated text.

Residual information exposed to the server or network path includes:

1. Request timing.
2. Ciphertext sizes.
3. Number of decode iterations.
4. Session identifiers and coarse traffic metadata.

These boundaries matter because they define what the architecture actually secures. The delivered system improves confidentiality against the server operator, but it is not a defense against all malicious-server, side-channel, or traffic-analysis attacks. It should therefore be understood as a practical privacy hardening of cloud-style inference rather than a universal secure-computation solution.

A second boundary is equally important for honest reporting: the current design does **not** guarantee model confidentiality against the client. The client receives local ONNX packages for non-encrypted blocks, decrypts intermediate projection outputs, and controls the query process. Relative to a conventional server-side API, this gives a determined client a richer basis for model extraction or approximation attacks [23], [24]. For that reason, provider-side model secrecy is treated as a non-goal of the present architecture.

Accordingly, the system is designed to protect **user data from the server**, not to provide simultaneous protection of **server-hosted model parameters against the client**. Any deployment that requires both goals would need additional mechanisms beyond the current hybrid FHE split.

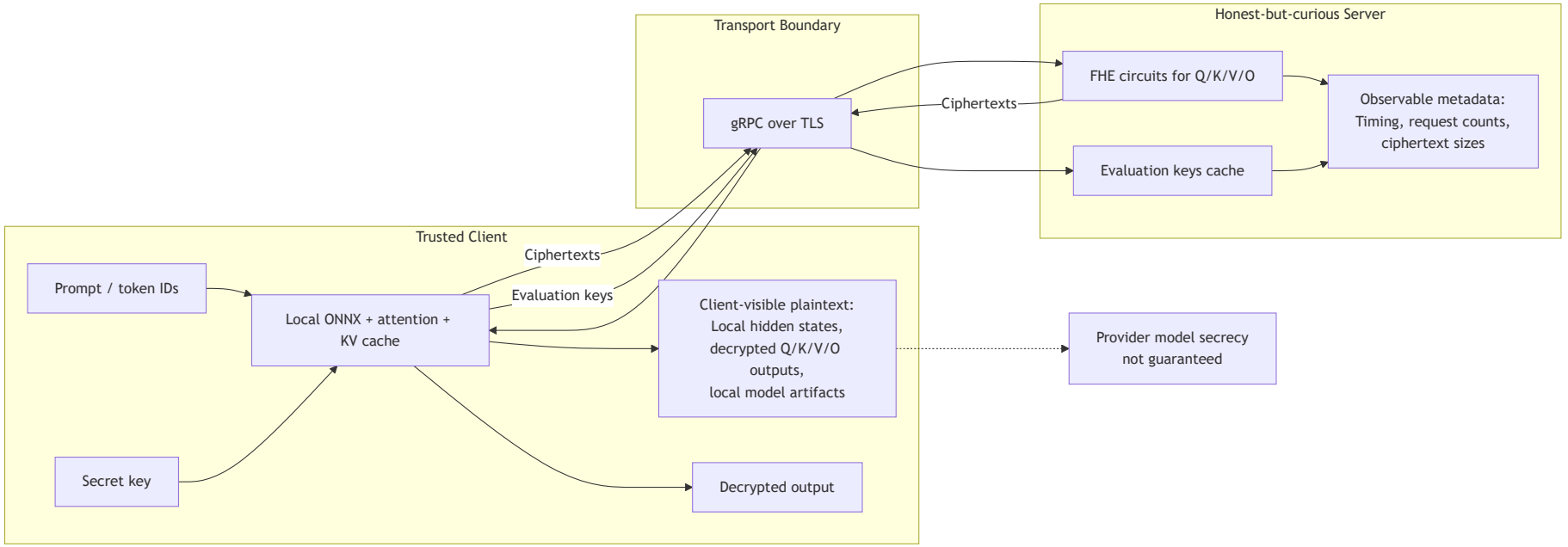

Figure 3.2: Threat model and trust boundaries

3.3 Key Design Decisions

The final architecture emerged through a sequence of trade-offs rather than from a single obvious design. Each major decision balanced privacy, server-side offload potential, non-linear FHE cost, runtime, and implementation complexity. Two decisions are especially important: the choice to make the runtime fully autoregressive rather than prefill-only, and the choice to calibrate deeper encrypted layers against the distributions they are likely to see after earlier encrypted layers have already introduced error.

Throughout the following discussion, a precision label such as `8/8/8` is ordered as `(q/k input bits, v/o input bits, weight bits)`. Accordingly, `6/6/8` means `q` and `k` use 6-bit inputs, `v` and `o` use 6-bit inputs, and the cleartext weights are quantized to 8 bits.

Table 3.1: Key design decisions and rationale

Design decision	Chosen option	Rationale
Model target	TinyLlama 1.1B	Large enough to be representative of a modern decoder-only LLM while remaining practical to compile and study within the project scope
Encrypted subgraph	Attention projections `q/k/v/o` only	Linear maps were the most practical FHE target under current tooling
Non-linear execution	Local on trusted client	SoftMax, masking, RMSNorm, and SwiGLU are costly or awkward under the same FHE budget
MLP under FHE	Deferred in this study	Additional circuits, longer compile/benchmark cycles, and deeper approximation mismatch would have expanded the validation burden beyond what could be completed carefully in one capstone
Runtime packaging	Compiler-generated reference-free artifacts	Client runtime is assembled from exported ONNX graphs, deployment bundles, and configuration rather than from the original checkpoint
Autoregressive execution	Explicit prefill plus token-by-token decode with KV cache	Validates the split in a real causal generation setting rather than in a one-step forward pass only
Local execution backend	Mixed ONNX Runtime and PyTorch local path	Most local graph execution uses ONNX Runtime, while attention remains in PyTorch because decode-time KV-cache handling and attention kernels are managed explicitly on the client
Transport	gRPC with binary bytes payloads	Avoids JSON/base64 overhead and fits large ciphertext transfers
Key handling	Session-based evaluation-key caching	Avoids repeated heavy key upload for every circuit call
Canonical precision	Uniform `8/8/8` (`q/k=8`, `v/o=8`, `weights=8`)	Best full-depth quality/latency trade-off in the final benchmark set
Calibration strategy	Curated prompts plus layer-aware cascaded calibration	Reduces calibration/inference mismatch across depth and between QKV and O-projection paths
Primary security goal	User prompt confidentiality against the server	Matches the honest-but-curious threat model implemented by the runtime
Provider-side model secrecy	Not guaranteed in current split	Client-visible intermediates and local model packages weaken confidentiality for the model owner

3.4 TinyLlama Architecture and Circuit Count

The project targets **TinyLlama 1.1B**, a compact causal decoder in the LLaMA family [13]. Only the architectural properties that materially affect the implementation are reproduced here.

Table 3.2: TinyLlama architecture specifications relevant to this project

Property	Value
Transformer layers	22
Hidden size	2048
Query attention heads	32
Key/value heads	4
Head dimension	64
Vocabulary size	32000
Attention pattern	Grouped-Query Attention
FHE projections per layer	`q`, `k`, `v`, `o`
Modes per projection	`prefill`, `decode`
Total compiled FHE circuits	`22 x 4 x 2 = 176`

Grouped-Query Attention is especially helpful here because only four key/value heads are maintained instead of thirty-two full key/value head sets. That reduces cache pressure, local reshaping overhead, and the amount of state that must be managed during decode-time execution.

3.5 Hardware and Software Components

Table 3.3: Hardware and software tools used in the project

Category	Item	Role in the project
Hardware	Local x86 CPU validation environment	Validation environment used for compilation and benchmark collection
Hardware	24 GB system memory	Available memory during compilation and benchmark collection
Runtime OS environment	Local WSL2/Python environment	Development and experiment execution environment
FHE stack	Concrete-ML 1.9.0, concrete-python 2.10.0	FHE compilation and deployment
Deep learning stack	PyTorch 2.3.1	Model inspection, compilation input, local attention kernels
Model libraries	Transformers 4.50.3	Tokenizer and optional reference-model benchmarking
Local graph execution	ONNX 1.16.1, ONNX Runtime 1.18.1	Local embedding, normalization, layer-body, and tail execution
RPC stack	gRPC / Protocol Buffers	Client-server transport and session API
Monitoring	Prometheus client	Server instrumentation
Analysis	CSV outputs and comparison scripts	Post-run metric aggregation and comparison

Table 3.3 records the environment used to validate the prototype and collect the benchmarks reported in Chapter 4. It describes the study environment rather than the architectural ceiling of the system. All Chapter 4 benchmarks and the later scaling projections were collected with the Concrete runtime left at its two-thread local setting, which is also recorded in the benchmark CSV files.

3.6 Offline Compilation and Packaging Workflow

The offline side of the system is best understood as a compiler and artifact packager rather than as a one-time preprocessing step. Starting from the base TinyLlama checkpoint, the project compiler produces the local ONNX blocks, the calibration corpora, the compiled FHE circuits, and the deployment metadata later consumed by the client and server runtimes. In the final evaluated configuration all 22 transformer layers are compiled for FHE attention projections, so the delivered artifact set corresponds to a full-depth hybrid runtime rather than a partial encrypted prefix.

In concrete terms, the compiler follows six top-level phases.

1. **Load the floating-point model.** TinyLlama and its tokenizer are loaded so that the compiler can export graph fragments and collect calibration activations from the original network.
2. **Export local and support ONNX assets.** The compiler writes the client-side ONNX family needed for reference-free execution: `embedding.onnx`, 22 input-norm ONNX

files, 22 layer-body ONNX files, and `final_tail.onnx`. It also exports per-layer `q/k/v/o` ONNX projection files for the FHE layers so that cascaded simulation and `o`-projection calibration reuse the same graph structure as deployment. An optional non-FHE projection path is retained for partial-split experiments, although that branch is inactive in the final 22-layer configuration.

3. **Export the full model once and import the graph.** A full ONNX checkpoint is created and then imported with graph-surgery tooling so that individual linear operators can be extracted precisely for `prefill` and `decode` compilation.
4. **Build separate prefill and decode calibration corpora.** Prefill calibration is collected from prompt-driven activations with matching attention masks. Decode calibration is built separately from real next-token embeddings and real KV caches captured from a true prefill pass, with explicit absolute position IDs so that the cached K tensors follow the same RoPE convention as runtime decode. Early percentile clipping is applied to keep ranges stable before deeper calibration stages.
5. **Run the per-layer compilation loop.** For each layer, the compiler updates the current calibration state, pre-extracts the required ONNX slices from the cached full graph, compiles the encrypted circuits, and advances to the next layer. Internally this phase already contains its own substructure: cascaded layer simulation, input-normalization alignment, six `q/k/v` compilations, `o`-projection calibration generation, and two `o` compilations. That inner loop is expanded in Section 3.7.
6. **Package, document, and verify deployment outputs.** After compilation, the compiler saves `calibration_data.npz`, `compilation_stats.json`, `client_assets.zip`, and `inference_config.json`, then verifies that every compiled `(layer, projection, mode)` tuple has both a `client.zip` and a `server.zip` artifact.

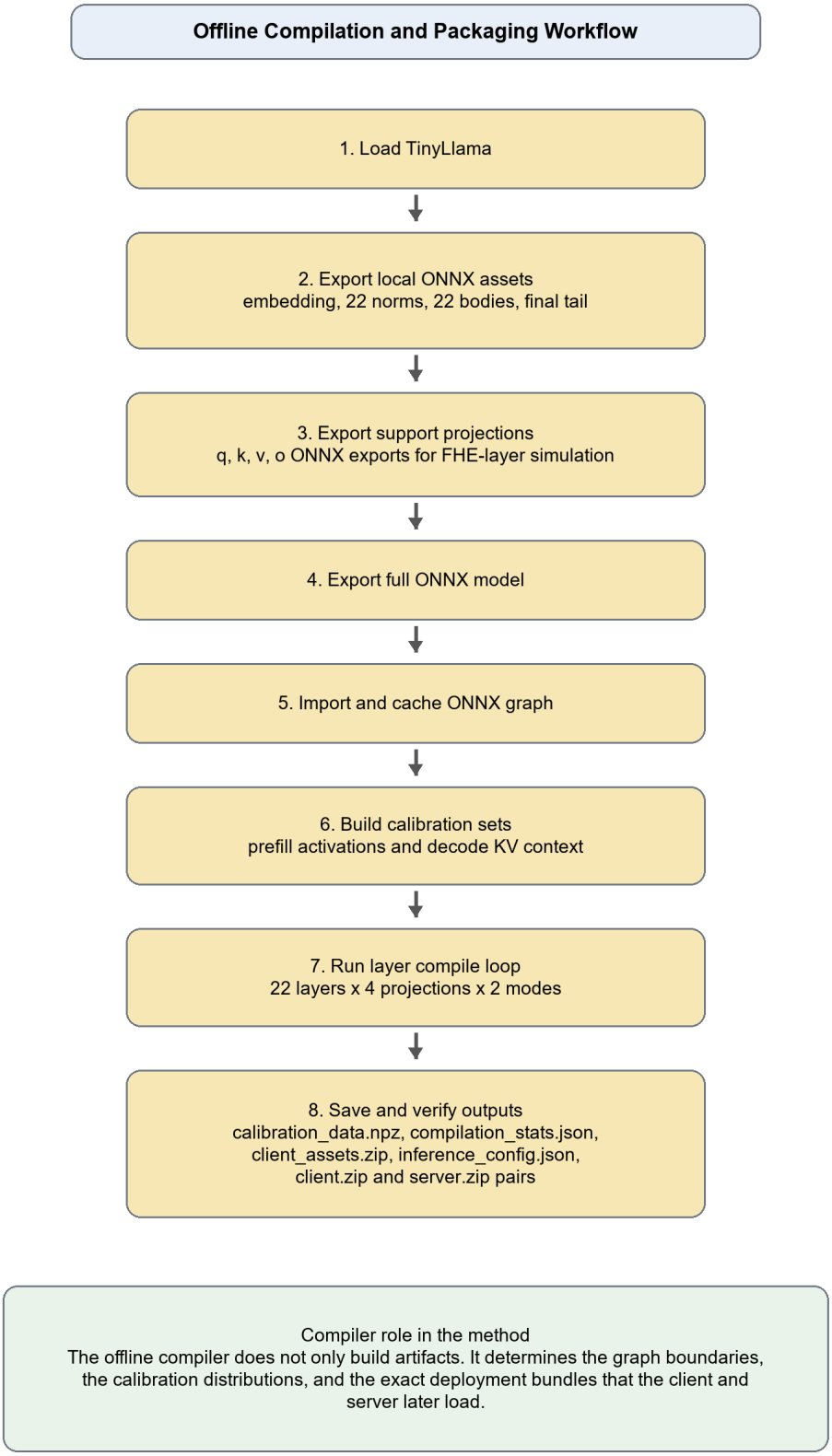

Figure 3.3: Offline compilation and packaging workflow

The resulting package is larger and more structured than a simple compiled-model dump. In the full-depth configuration it contains one embedding graph, 22 input norms, 22 layer bodies, one final tail, 88 exported projection ONNX files used for calibration and simulation, and 176 compiled FHE circuits with matching client/server deployment bundles. Because the runtime is assembled directly from these outputs, the compiler is part of the research method rather than merely a build convenience.

The same workflow also shapes later communication results. During circuit compilation, the deployment interface enables input-ciphertext compression and evaluation-key compression where those options are available, which helps on the client-to-server path. An equivalent practical mechanism for compressing returned ciphertexts was not available in the workflow used here, which helps explain why download cost remains dominant in Chapter 4.

3.7 Per-Layer Calibration and FHE Compilation Loop

Figure 3.3 still compresses the hardest part of the compiler. The key methodological choice is to interleave calibration and circuit generation layer by layer so that deeper circuits see increasingly realistic input distributions after earlier encrypted layers have already introduced error.

For each transformer layer, the inner loop proceeds as follows.

1. **Choose the current calibration state.** Layer 0 uses the clean prefill and decode calibration sets. For deeper layers, the compiler can first simulate the complete previous layer so that the next layer is calibrated on encrypted-like upstream activations rather than idealized floating-point hidden states.
2. **Align calibration with the true Q/K/V inputs.** Because the extracted `q/k/v` ONNX slices are bare linear operators, the compiler applies the exported input LayerNorm ONNX graph to the current calibration tensors before compiling those circuits. Without this step, the compiler would quantize against a distribution that inference never actually sends.
3. **Compile Q/K/V for both runtime modes.** The compiler pre-extracts six ONNX slices per layer (`q/k/v` times `prefill/decode`) from the cached full graph and compiles them with mode-specific calibration data. Layer-specific bit-width selection is applied here, and the same compile step also applies the chosen `p_error` plus input-ciphertext and evaluation-key compression where the deployment interface exposes them.
4. **Generate O-projection calibration from the quantized attention path.** After `q/k/v` are available, the compiler loads the corresponding deployment clients, reuses their quantizers, and runs the quantized outputs through the local attention path to form a more realistic calibration set for `o`. This is done separately for prefill and decode, with real KV caches and decode-time masks used in the decode branch.
5. **Compile O for both runtime modes and advance.** The compiler extracts the two `o` slices (`prefill` and `decode`), compiles them against the newly generated `o` calibration tensors, stores the artifacts, and advances to the next layer.

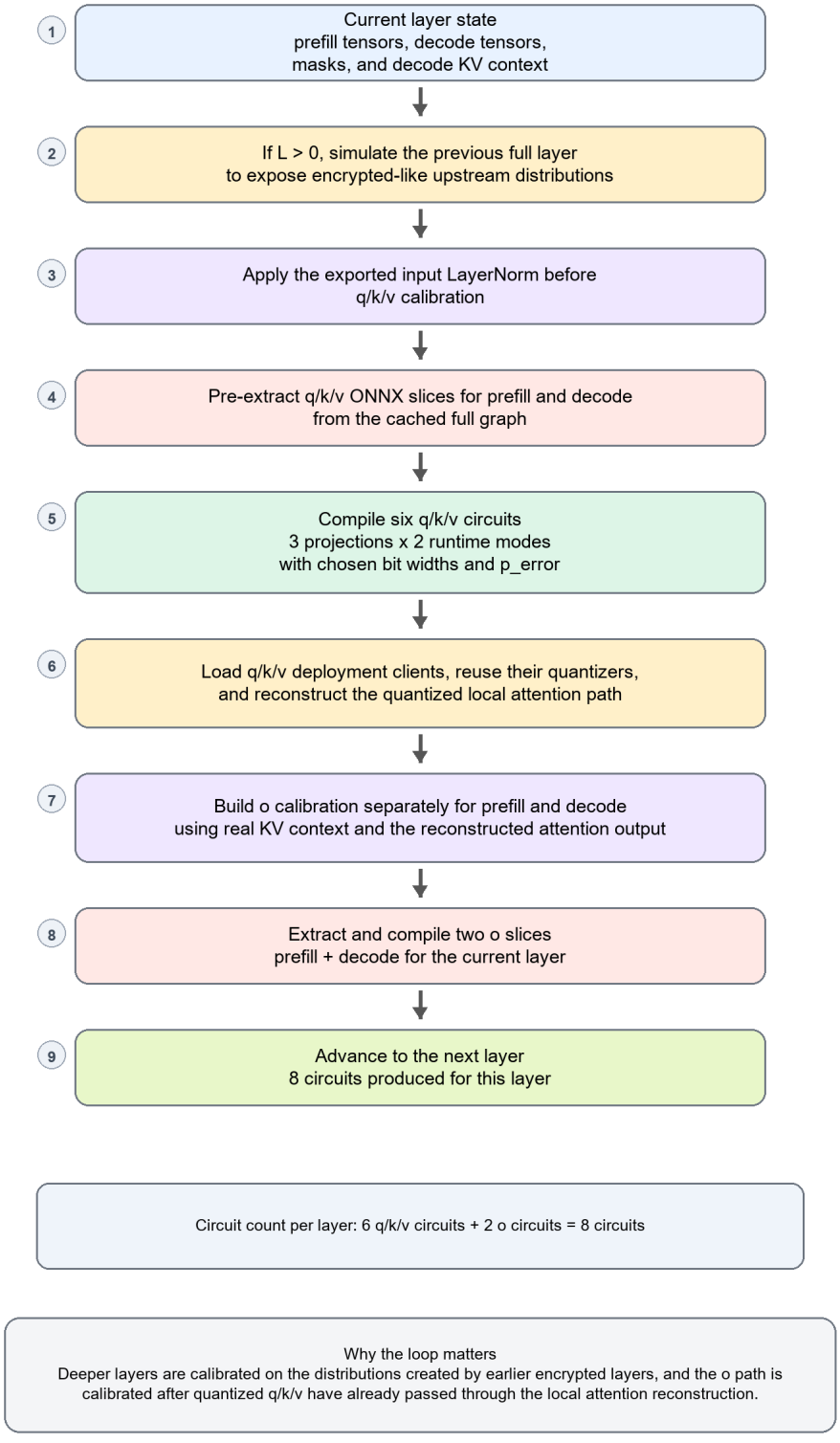

Figure 3.4: Per-layer calibration and FHE compilation loop

Each layer therefore yields **eight circuits**: q, k, v, and o, each compiled once for prefill and once for decode. Across 22 layers that produces the final 176-circuit deployment. More importantly, the loop couples four calibration controls: separate prefill/decode data, real-KV decode context, cascaded layer-to-layer simulation, and distinct o-path calibration after quantized QKV. The key data dependency is the handoff from quantized q/k/v outputs to o calibration through the reconstructed local attention path, so the o circuits see the same kind of distorted attention context they will later receive at runtime. That coupling, rather than any single compile call, is what keeps the full-depth runtime stable enough to evaluate.

3.8 Client Architecture

The client is built from compiler outputs rather than from the original checkpoint. At startup it loads inference_config.json, ONNX Runtime sessions for the exported local blocks, and one FHEModelClient instance for each compiled circuit. The tokenizer still comes from the Transformers library, while PyTorch remains in the loop for local attention operations such as scaled_dot_product_attention.

Here, "reference-free" means that the deployed client no longer depends on the original TinyLlama weights at runtime, although it is not an ONNX-only implementation. In the final full-depth configuration, ONNX Runtime handles embedding, 22 input norms, 22 layer bodies, and the final tail. The codebase also retains local q/k/v/o ONNX sessions for partial-split experiments, although that branch is not used in the final 22-layer benchmark.

The client carries the autoregressive state across the split. After prefill, each selected token is embedded locally, appended to the attention mask, assigned its next rotary position, merged into the KV cache, and sent back through the same 22-layer hybrid path. The client therefore has to preserve decode state correctly while repeatedly leaving and re-entering the encrypted server path.

Two implementation choices were especially important in practice. First, evaluation keys are uploaded once per session and then reused across circuit calls. The client can query session status, refresh an expiring session, and fall back to per-request key transmission if session caching is unavailable. Second, q/k/v are treated as a distinct communication path: the client first tries a server-streaming QKV RPC, then a non-streaming batched RPC, and only then falls back to three separate projection calls. Because QKV dominates the number of server interactions, these transport choices matter in repeated decode loops.

The streaming path is also important because it changes how work overlaps inside the loop. The server computes q, k, and v concurrently and returns each ciphertext as soon as it is ready. The client decrypts each result immediately instead of waiting for the full batch to arrive. This does not alter correctness, but it overlaps part of client-side decryption with the tail of server-side computation and trims idle time from the decode path. The optional HuggingFace model remains confined to benchmarking, so the deployed split itself stays reference-free.

The client is responsible for:

1. Tokenization and prompt preparation.
2. Loading compiled ONNX assets and per-circuit FHE client artifacts.
3. Session setup and evaluation-key upload.
4. Local ONNX execution for embedding, normalization, layer body, and final tail.
5. Local RoPE handling and KV-cache maintenance.
6. Local attention score computation and attention masking.
7. Prefill/decode control flow and next-token selection.
8. Decryption of returned ciphertexts.
9. Benchmark measurement and structured result export.

In addition to inference control, the client is also the measurement point for the experiments. It computes semantic similarity, token accuracy, first-token accuracy, BLEU, perplexity difference, communication totals, and the simple idealized server projections used later in the analysis chapter. Because the client also timestamps encryption, decryption, network, and server-compute contributions separately, later analysis can break latency into phase-resolved components instead of reporting only end-to-end time.

3.9 Server Architecture

The server has a deliberately narrow role. It loads `inference_config.json`, maps each compiled `(layer, projection, mode)` tuple to the corresponding `server.zip` deployment artifact, accepts ciphertext bytes, executes the requested circuit, and returns ciphertext bytes. It does not receive the secret key, and under the intended threat model it should never observe plaintext prompts or plaintext hidden states.

That narrowness is one of the design strengths of the system. By keeping the server focused on circuit execution, session-cached evaluation keys, and RPC orchestration, the report can discuss trust boundaries, protocol state, and communication cost without mixing them with local transformer logic.

At the same time, the server is more than a thin wrapper around `FHEModelServer.run()`. Evaluation keys are cached per circuit inside each session rather than globally. The server tracks session expiry, runs background cleanup, and enforces a bounded session capacity so that long decode runs do not accumulate state without control. On the execution side, it exposes three projection paths: a single-layer endpoint, a batched QKV endpoint, and a server-streaming QKV endpoint that emits results in first-completed order. Internally, per-circuit locks protect FHE execution while a persistent worker pool handles QKV concurrency. These are engineering choices rather than algorithmic novelties, but they have a direct effect on robustness and runtime practicality.

The server also includes several control-plane features that make the prototype easier to operate and study: TLS support, optional API-key checking, health/config/statistics endpoints,

and Prometheus metrics. Together they make the prototype closer to an operational encrypted-inference service skeleton than to a one-off experiment script.

Table 3.4: Server-side components

Component	Function
`HealthCheck` / `GetConfig` / `GetStatistics`	Server introspection and monitoring support
`InitSession` / `GetSessionStatus` / `DeleteSession`	Evaluation-key upload and session lifecycle
`ProcessLayer`	Single encrypted projection request
`ProcessQKVBatch`	Batched QKV computation
`StreamProcessQKVBatch`	Progressive QKV streaming for earlier client decryption
`ProcessBatch`	Generic multi-request processing
TLS configuration	Protected transport, with self-signed generation support for development
API-key interceptor	Authenticates gRPC metadata when enabled
Prometheus instrumentation	Exposes runtime metrics
Thread pools	Separate pools for request handling and QKV batch work

Because the client keeps decode state while the server only evaluates encrypted projections, the runtime is best understood as a repeated interaction pattern rather than a single monolithic forward pass. Figure 3.5 summarizes that prefill/decode exchange.

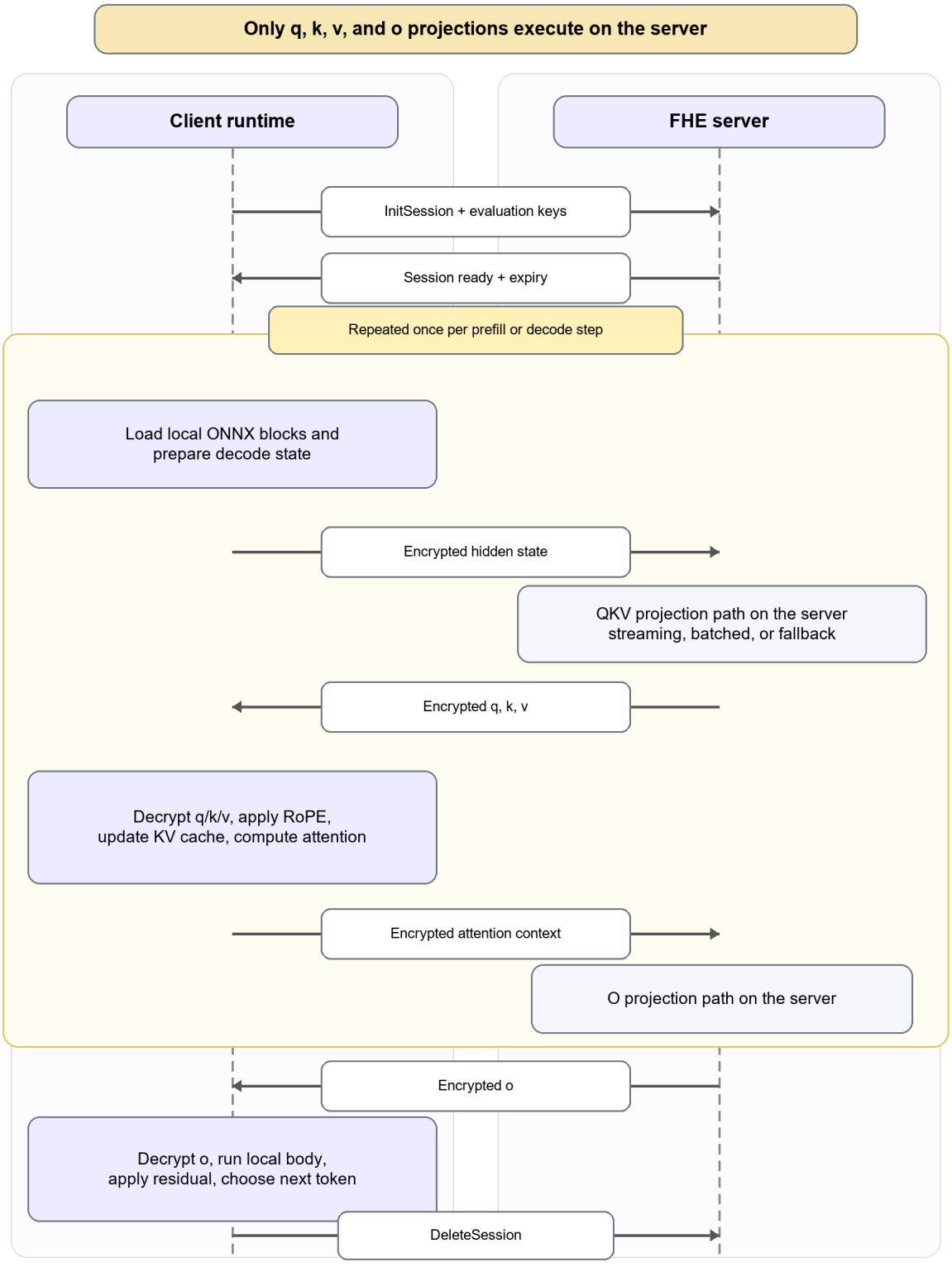

Figure 3.5: Client-server interaction during hybrid inference

3.10 Communication Design and Compression Constraint

Communication design is not a secondary implementation detail in this project. Because FHE ciphertexts are large, transport choices directly affect both feasibility and interpretation of the results. In an ordinary application, serialization format might be an implementation convenience. Here it becomes part of the research result.

First, the transport was moved to **gRPC binary bytes** rather than JSON/base64. This is a straightforward systems decision: FHE ciphertexts are already large, and textual serialization overhead is wasteful.

Second, the project explicitly investigated communication overhead mitigation. Earlier transport experiments in this project showed that gRPC gzip compression did not materially help on FHE ciphertexts and added CPU overhead. Section 4.11 later quantifies this on matched two-layer benchmark families. The result is consistent with the intuition that ciphertext bytes are not very compressible through naive transport-layer compression.

Third, the current compiler/deployment path matters directly. Circuit compilation enables input-ciphertext compression and evaluation-key compression where those options are exposed by the deployment API. Those settings help on the upload side. However, the Concrete-ML client-server workflow used in this project did not expose a comparable practical output-ciphertext compression API for returned encrypted results. As a result, returned ciphertexts dominate communication volume. This does **not** mean output compression is impossible in principle. It means that, in the specific deployment workflow used here, that lever was not yet directly available to the system.

3.11 Implementation Refinement and Validation

The project evolved through repeated correction rather than through a single clean implementation pass. In a hybrid encrypted system, small mistakes in masking, positional encoding, decode-state handling, graph extraction, or calibration can easily be misread as cryptographic degradation. The implementation therefore had to be stabilized before the benchmark results could be treated as trustworthy evidence. Several correctness-oriented refinements were especially important:

1. Better alignment of RoPE, explicit position handling, and left-padding across compile-time calibration and runtime inference.
2. Correct treatment of RMSNorm epsilon from model configuration.
3. Decode-mask and KV-cache fixes so that decode calibration and decode inference use consistent sequential context.
4. Input-LayerNorm alignment before Q/K/V calibration, so extracted linear slices see the same input distribution at compile time and runtime.
5. Calibration/inference alignment improvements, including real-KV decode calibration,

cascaded layer-wise calibration, and separate o-projection calibration from quantized attention outputs.

6. Better packaging for reference-free runtime components and verification of all compiled artifacts.
7. Migration from heavier textual transport to raw binary gRPC payloads with session-based key reuse.

These refinements matter because output quality in a hybrid encrypted system is sensitive not only to cryptographic effects, but also to padding, masking, positional encoding, graph extraction boundaries, decode-state consistency, and calibration correctness. For that reason, software validation is treated here as part of the methodology rather than as a separate engineering afterthought.

Chapter 4

Investigation/Experiment, Result, Analysis and Discussion

4.1 Experimental Setup

The experimental record for this project includes sixty benchmark logs collected across debugging sessions, ablation studies, and later full-depth evaluations. They do not all carry the same evidentiary weight. Early runs were useful for finding implementation errors and stabilizing the pipeline, but they are not equally suitable as headline results. This chapter therefore relies on a smaller, curated subset of runs chosen to answer concrete questions about encrypted depth, quantization, calibration, generation length, and communication cost.

The primary benchmark used throughout the chapter is the complete 22-layer evaluation with uniform `8/8/8` quantization and a maximum of five new tokens per prompt. It is treated as the main reference point for four reasons:

1. It represents the intended final architecture rather than an intermediate checkpoint.
2. It covers all encrypted layers instead of a shallow subset.
3. It belongs to a family of nearby ablations that make its behavior interpretable.
4. It provides the strongest full-depth balance of quality and practicality in the available results.

The precision notation used in this chapter follows the compiler configuration directly: `q/k` input bits, `v/o` input bits, and weight bits. Thus `8/8/8` means `q` and `k` use 8-bit inputs, `v` and `o` use 8-bit inputs, and the weights are quantized to 8 bits; `6/6/8` is defined analogously.

The prompt set contains ten short prompts spanning factual recall, coding, open-ended completion, and general assistant-style language. The main benchmark uses a five-token generation cap per prompt so that full-depth evaluation remains computationally manageable. A separate 20-token-cap stress test is then used to study error accumulation in the fully autoregressive decode loop.

The plaintext baseline is the same model family executed without FHE. All reported quality

metrics are computed automatically by the client after each benchmarked prompt. The runtime itself is fully autoregressive: each generated token is fed back into the next decode step, each step updates the KV cache and position state, and each step requires encrypted `q/k/v/o` calls across all 22 layers. The experiments therefore evaluate more than whether encrypted projections can be invoked once. They test whether a deep hybrid causal runtime can remain coherent across repeated decode iterations.

The experiment families in this chapter are also tied directly to the compiler and runtime levers introduced in Chapter 3. Layer-count and full-depth bit-width studies test how far the compiled encrypted subgraph can be pushed. The adaptive early-layer study tests whether front-loading `q/k` and `v/o` precision is enough to stabilize later depth. The calibration-source and cascaded-calibration comparisons test the compiler-side data and method choices summarized in Figures 3.3 and 3.4. The longer-generation study tests whether the fully autoregressive runtime remains stable once the decode loop is allowed to run further. The shallow `p_error` and gzip comparisons, finally, are not headline benchmarks; they are parameter-selection and systems-screening experiments used to decide which deeper runs were worth pursuing.

The benchmark record is also reproducible in a practical engineering sense. The retained CSV files log prompt text, token counts, local thread count, timing components, communication totals, call counts, and simple scaling fields for each run. The compiler configuration, calibration corpus, and packaged deployment artifacts described in Chapter 3 and Appendix D are retained alongside those logs, so the benchmark families can be reconstructed from the project materials.

4.2 Evaluation Metrics

Table 4.1: Evaluation metrics

Metric	Meaning	Why it is useful here
Semantic similarity	Cosine similarity between reference and FHE outputs using sentence embeddings	Captures rough semantic drift
Token accuracy	Fraction of matching generated tokens	Direct generation agreement measure
First-token accuracy	Whether the first generated token matches	Useful because early decode errors strongly affect later outputs
BLEU	N-gram overlap between plaintext and FHE outputs	Complements semantic similarity with surface-form agreement
Regular PPL / FHE PPL	Perplexity under plaintext and FHE outputs	Measures fluency/confidence degradation
PPL ratio	`FHE_PPL / Regular_PPL`	Normalized quality-loss indicator
FHE time	End-to-end encrypted inference time	Practicality indicator
Server compute / client encrypt / client decrypt / network / ONNX	Latency decomposition	Identifies dominant bottlenecks
Upload / download bytes	Communication overhead	Important for remote deployment feasibility

No single metric fully characterizes encrypted generation quality. Semantic similarity can remain moderate even when specific factual tokens are wrong, while token accuracy is highly sensitive to surface-form drift. The results are therefore interpreted through several metrics together rather than through any one score in isolation.

4.3 Headline Benchmark

Table 4.2: Headline benchmark summary

Item	Value
Model	TinyLlama 1.1B
Encrypted layers	22
Encrypted projections	`q`, `k`, `v`, `o`
Quantization	`q/k=8`, `v/o=8`, `weights=8`
Maximum new tokens	5
Prompts evaluated	10
Average semantic similarity	0.4852
Average token accuracy	0.4833
Average first-token accuracy	0.6000
Average BLEU	0.3451
Average regular PPL	3.2517
Average FHE PPL	4.4259
PPL ratio	1.3611x
Average end-to-end FHE time	3396.285 s
Average prefill time	2806.446 s
Average decode time per token	154.779 s
Encrypted projection calls for 5-token generation	440
Average total communication	19261.9687 MB

The headline benchmark marks the limit reached by the current implementation. It is too expensive for routine deployment, yet it is far from a failure case in which encrypted generation collapses into unusable text. Across the prompt set, the model still preserves recognizable structure while keeping the prompt hidden from the server. The benchmark used a five-token cap, and a full-length completion at that cap requires 440 encrypted projection calls across the 22-layer runtime. One prompt stopped after three new tokens, so the realized prompt-average call count in the CSV is lower. Even with that early stop, the benchmark still reflects sustained causal execution rather than a single encrypted forward pass.

Because the headline result is averaged over only ten prompts, the means should be interpreted alongside the prompt-level spread. Semantic similarity has a standard deviation of `0.3112` and ranges from `0.1702` to `1.0000`. Token accuracy has a standard deviation of `0.3375` and ranges from `0.1667` to `1.0000`. BLEU has a standard deviation of `0.4031` and ranges from `0.0000` to `1.0000`. End-to-end FHE time has a standard deviation of `204.037 s` and ranges from `2924.67 s` to `3570.10 s`. These numbers describe feasibility over a small prompt set, not low-variance production behavior.

4.4 Effect of Encrypted Layer Count

The first major ablation asks a basic systems question: what happens as more of the model is pushed behind the FHE boundary? The comparison uses the `6/6/8` family because it provides a reasonably consistent progression from shallow to full-depth runs.

Table 4.3: Effect of encrypted layer count

FHE layers	Similarity	Token acc.	First-token acc.	BLEU	PPL ratio	FHE time (s)
1	0.8475	0.8000	0.9000	0.6419	1.0841	133.137
3	0.5601	0.5667	1.0000	0.4162	1.0309	420.616
4	0.5150	0.4833	0.9000	0.3381	1.1363	575.869
6	0.4801	0.4333	0.8000	0.3040	1.2308	840.337
11	0.2961	0.2593	0.4444	0.1156	1.4188	1532.089
22	0.2952	0.2833	0.6000	0.1584	2.0931	3033.311

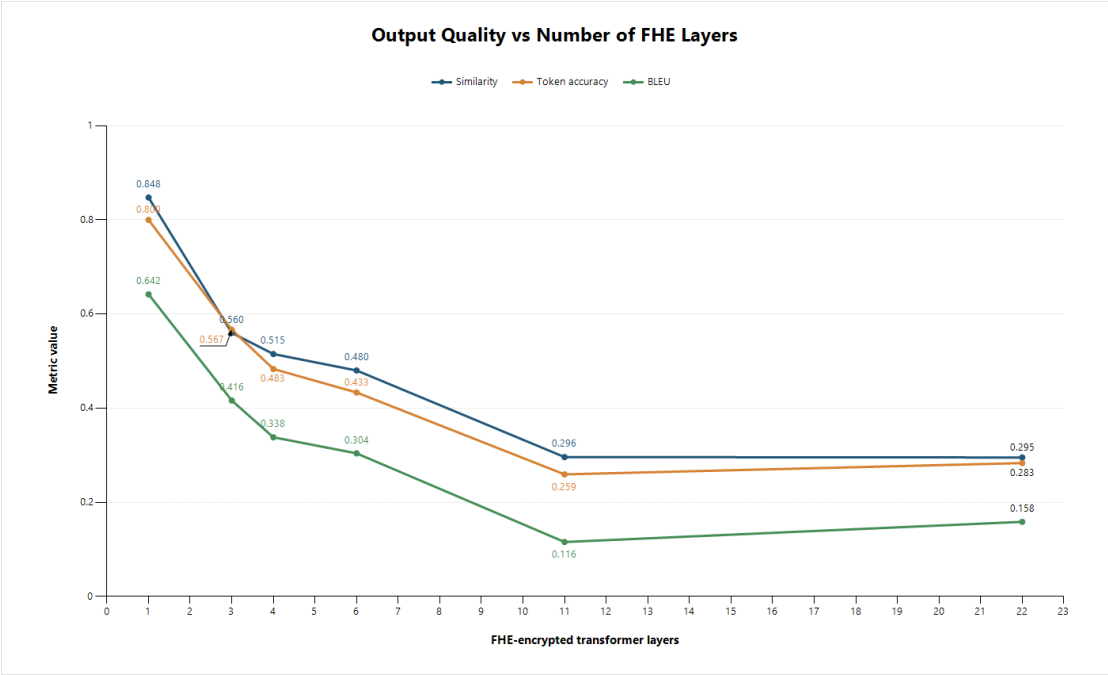

Figure 4.1: Output quality versus number of encrypted layers.

Three trends are clear from this progression.

1. Quality degrades as encrypted depth increases. This is expected because more encrypted layers mean more opportunities for quantization mismatch and ciphertext noise to perturb the hidden-state trajectory.
2. The degradation is not perfectly linear. Similarity falls sharply from one to three layers and then continues to decline more gradually, which suggests that early encrypted distortions propagate forward through later layers.

3. Runtime scales upward with depth. Moving from one encrypted layer to twenty-two increases end-to-end time by more than an order of magnitude.

This experiment justifies the project's central architectural compromise. Encrypting more of the model increases the scope of server-side confidentiality, but it also pushes the system deeper into a regime where both quality and latency become harder to manage.

4.5 Effect of Bit-Width at Full Depth

The next experiment keeps depth fixed at 22 layers and varies quantization precision. This isolates a different question: given the chosen hybrid architecture, what full-depth precision setting offers the best trade-off?

Table 4.4: Effect of bit-width at 22 layers

Config	Similarity	Token acc.	First-token acc.	BLEU	PPL ratio	FHE time (s)
`6/6/8` (`q/k=6`, `v/o=6`, `weights=8`)	0.2952	0.2833	0.6000	0.1584	2.0931	3033.311
`8/8/8` (`q/k=8`, `v/o=8`, `weights=8`)	0.4852	0.4833	0.6000	0.3451	1.3611	3396.285
`10/10/10` (`q/k=10`, `v/o=10`, `weights=10`)	0.4203	0.4500	0.6000	0.3100	1.3488	3518.822

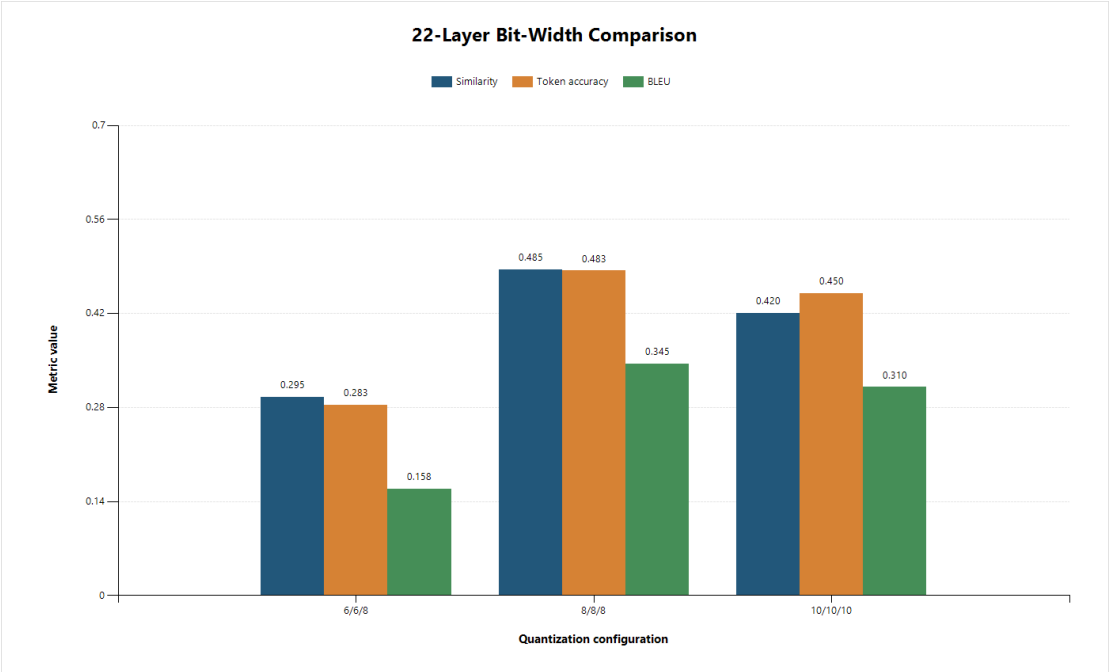

Figure 4.2: Comparison of 22-layer bit-width configurations.

The most important conclusion from this comparison is that **uniform `8/8/8` is the strongest full-depth operating point in the current study**.

The behavior of the three settings is informative:

1. `6/6/8` is faster, but the quality drop is too large. Similarity and BLEU both fall sharply, and perplexity ratio more than doubles relative to plaintext.

2. 8/8/8 recovers a large amount of quality with only a moderate time increase.
3. 10/10/10 does not continue the quality gains in a clean monotonic way. Although the perplexity ratio remains close to the 8/8/8 case, similarity and BLEU are slightly worse and runtime is slightly longer.

This non-monotonic pattern explains why empirical tuning was necessary. More bits do not automatically yield a better encrypted transformer pipeline once compilation limits, runtime cost, and distribution mismatch are taken into account.

The three full-depth runs compared here share the same 22-layer architecture, the same ten-prompt benchmark set, and the same five-token cap. However, the saved artifacts do not prove that every compile-time toggle beyond bit width was identical across the three runs. The comparison is therefore best treated as a same-family operating-point study rather than as a perfectly controlled A/B/C experiment.

The 10/10/10 configuration did not improve quality monotonically over 8/8/8, even though it used higher precision. A possible explanation is accumulator-range pressure or a related numerical effect at higher input precision. However, the experiments conducted in this project do not isolate that mechanism directly. For that reason, the weaker 10/10/10 result is reported here as an empirical observation rather than as confirmed overflow.

A separate shallow screening experiment was used to choose the circuit error budget. In the available two-layer pair, moving from p_error = 0.01 to p_error = 0.1 produced identical generated continuations on all ten prompts. Average similarity, token accuracy, first-token accuracy, and BLEU remained 0.6497, 0.5667, 0.8000, and 0.4234, respectively. Average FHE perplexity changed only from 3.2372 to 3.2377, or about +0.014%, while average FHE time fell from 269.344 s to 250.977 s (-6.82%), average server compute fell from 206.154 s to 191.197 s (-7.26%), and average total communication fell from 1574.560 MB to 1540.012 MB (-2.19%). On that basis, p_error = 0.1 was kept as the practical default for the deeper experiments that followed. This result supported shallow parameter selection only; it does not show that p_error is unimportant at full depth.

4.6 Adaptive versus Uniform Precision

The adaptive experiments were not arbitrary mixed-precision trials. They were a specific progression: start from a full-depth 6/6/8 baseline, then progressively replace the earliest layers with 8/8/8, since early-layer distortion propagates through every later layer. The sequence therefore moves from 8/8/8 on layers 0-3, to layers 0-7, to layers 0-10, and finally to all 22 layers.

Table 4.5: Adaptive versus uniform precision strategies

Strategy	Similarity	Token acc.	First-token acc.	BLEU	PPL ratio	FHE time (s)
Layers 0–3 at 8/8/8, layers 4–21 at 6/6/8	0.4185	0.4333	0.7000	0.2016	1.4344	3388.923
Layers 0–7 at 8/8/8, layers 8–21 at 6/6/8	0.4115	0.5333	0.6000	0.3047	1.3998	2895.404
Layers 0–10 at 8/8/8, layers 11–21 at 6/6/8	0.4658	0.4833	0.7000	0.2880	1.4014	7086.123
Layers 0–21 at 8/8/8	0.4852	0.4833	0.6000	0.3451	1.3611	3396.285

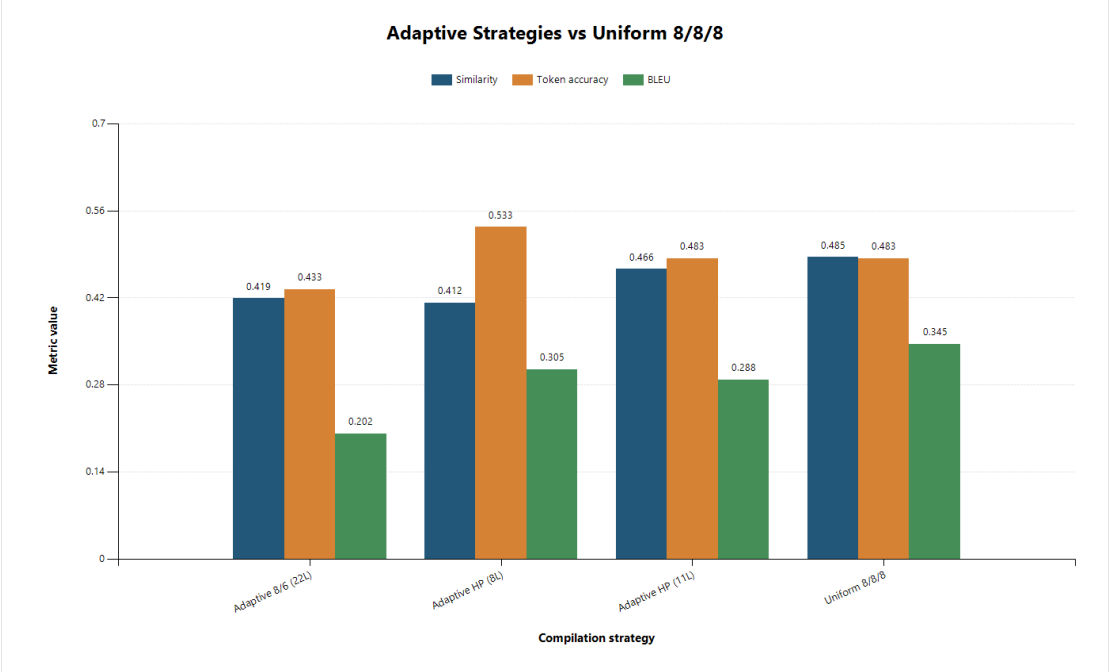

Figure 4.3: Adaptive strategies versus uniform 8/8/8.

This progression reveals a diminishing-returns pattern, but not the same pattern on every metric.

1. Moving from four early high-precision layers to eight produces the clearest practical gain in token-level agreement. Token accuracy rises from 0.4333 to 0.5333, BLEU rises from 0.2016 to 0.3047, and the perplexity ratio improves modestly from 1.4344x to 1.3998x.
2. Moving from eight to eleven high-precision layers is mixed rather than uniformly better. Similarity rises from 0.4115 to 0.4658, but token accuracy falls back to 0.4833 and BLEU slips to 0.2880.
3. Moving from eleven high-precision layers to full 8/8/8 produces further but smaller improvement. Similarity rises to 0.4852, BLEU rises to 0.3451, and perplexity ratio improves to 1.3611x, while token accuracy stays at 0.4833.

The 11-layer runtime line is dominated by a single extreme outlier prompt (44656.85 s for *The function returns*), while the median runtime of the same file is only 2912.83 s. For that reason, the 11-layer experiment is more informative as a quality study than as a clean latency point.

The main lesson is that upgrading the first several layers matters much more than upgrading every additional layer at the same rate. Even so, the simplest full-depth operating point remains the uniform `8/8/8` configuration, which avoids layer-specific policy logic while still delivering the best overall quality in the final benchmark family.

4.7 Calibration Strategy and Source Effects

Calibration quality turns out to be one of the strongest hidden variables in the entire pipeline. In this project, calibration matters at two different levels: the **source distribution** used to calibrate the circuits, and the **layer-wise method** used to propagate calibration across depth.

Table 4.6: Calibration source comparison

Calibration source	Similarity	Token acc.	First-token acc.	BLEU	PPL ratio	FHE time (s)
Curated prompt set	0.4852	0.4833	0.6000	0.3451	1.3611	3396.285
WikiText-2	0.2806	0.2833	0.5000	0.1280	3.4543	3134.651

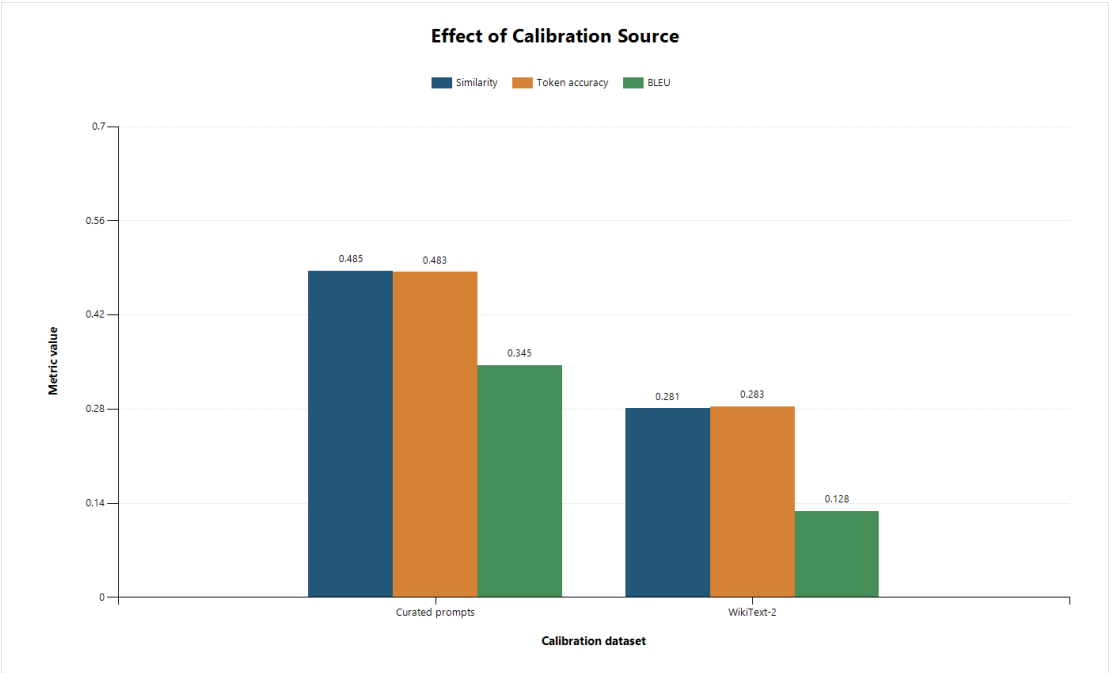

Figure 4.4: Effect of calibration source on quality.

The gap between the two settings is large enough to show that calibration is not a minor preprocessing detail. In this system it behaves as a first-order design variable. A calibration set that resembles the actual inference workload preserves far more of the model's behavior than a mismatched corpus, even when the quantization recipe and encrypted depth remain otherwise comparable.

That observation matters beyond this one benchmark. In an encrypted deployment pipeline, ”compilation” is not merely a technical translation step from model to circuit. It is also an act of distribution matching. If the calibration data poorly reflects the inputs seen at inference time, the deployed system begins in a numerically disadvantaged state before the first prompt is even processed.

The second calibration question is methodological rather than data-centric. The compiler does not only collect calibration prompts. It also implements a **cascaded, layer-aware calibration procedure** in which deeper layers can be calibrated on simulated noisy output from earlier encrypted layers, together with separate `o`-projection calibration derived from quantized attention outputs and percentile clipping to prevent range explosion. In methodological terms, this is the inner workflow illustrated by Figures 3.3 and 3.4, not an afterthought added after compilation. That is important because a deep encrypted stack does not receive the same distribution at layer 10 that it would see in a clean floating-point forward pass.

The nearest same-family comparison is between two full-depth `8/8/8` benchmark families, one with cascaded calibration enabled and one produced after disabling that toggle. The comparison is informative even if it is not a perfect textbook A/B: with cascaded calibration enabled, average similarity is `0.4852`, BLEU is `0.3451`, average FHE perplexity is `4.4259`, and runtime is `3396.285 s`. With the toggle disabled, similarity falls to `0.4777`, BLEU falls to `0.2651`, average FHE perplexity worsens to `5.1805`, token accuracy remains `0.4833`, and runtime improves to `2970.715 s`. Read proportionally, disabling cascaded calibration saves roughly one-eighth of runtime but costs nearly one-quarter of BLEU and about one-sixth of average FHE perplexity.

Within the available same-family comparison, cascaded calibration trades additional runtime for better distribution matching and better retained generation quality at full depth. That is strong evidence that the method matters in this system, and it justifies treating layer-aware cascaded calibration as one of the main method contributions of the project. The evidence is specific to the present pipeline and should not be generalized automatically to all encrypted LLM systems.

4.8 Longer Autoregressive Generation

Longer decode loops are harder because each generated token becomes part of the context for future tokens. Any early error can therefore propagate forward and distort the hidden-state trajectory of later steps. This is especially important here because the system is fully autoregressive: after prefill, every new token triggers another decode pass through all 22 encrypted layers.

This experiment verifies that the implementation is operating as a genuine causal generator rather than as a single-pass encrypted layer pipeline. A system that performs only one encrypted forward pass can avoid many of the difficult decode-time issues. The present system cannot.

Each extra token extends the KV cache, advances rotary positions, issues another full sequence of encrypted projection calls, and feeds the selected token back into the next step.

Table 4.7: Longer-generation comparison

Setting	Similarity	Token acc.	First-token acc.	BLEU	PPL ratio	FHE time (s)
5-token-cap benchmark	0.4852	0.4833	0.6000	0.3451	1.3611	3396.285
20-token-cap stress test	0.3275	0.2222	0.6667	0.1539	1.7487	4852.645

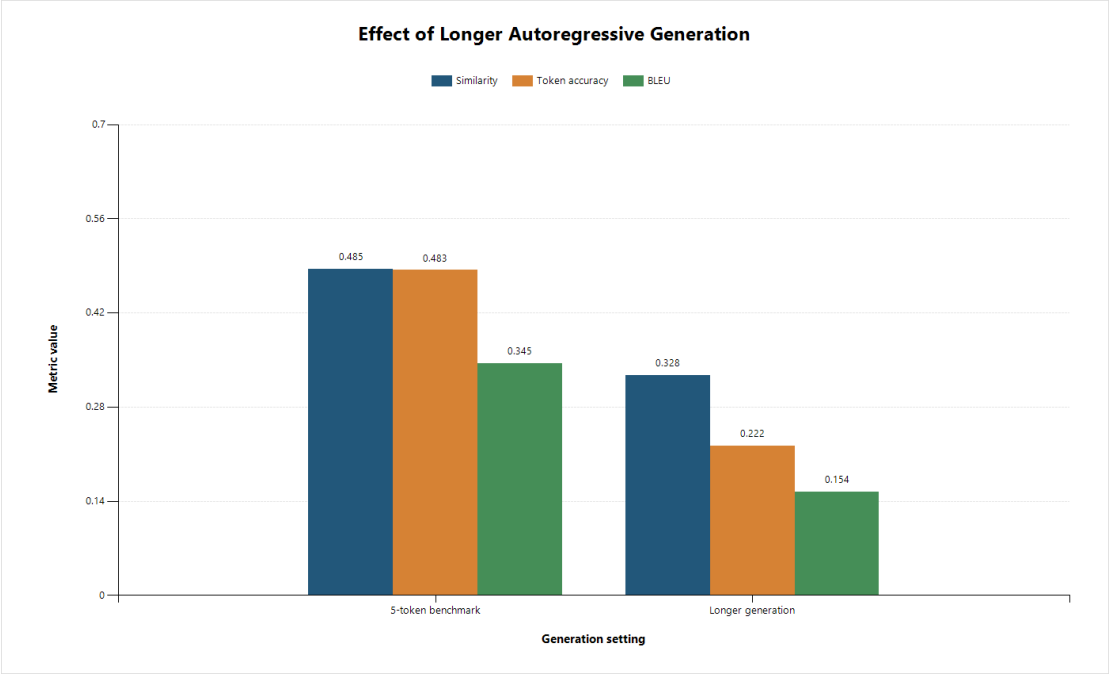

Figure 4.5: Effect of longer autoregressive generation.

The longer run should be described carefully. It is a 20-token-cap experiment, not a guaranteed 20-token output on every prompt; the six evaluated prompts produced an average of 15.0 generated tokens before stopping. Even with that qualification, the degradation pattern is clear: similarity falls by 32.5%, token accuracy by 54.0%, BLEU by 55.4%, and perplexity ratio rises by 28.5% relative to the five-token benchmark. This is strong evidence of autoregressive error accumulation rather than a one-step approximation effect. It also strengthens the claim that the delivered artifact is a genuine causal generator, not a static encrypted layer demo with language-model vocabulary attached afterward.

4.9 Per-Prompt Qualitative Analysis

Average metrics are useful, but they hide how uneven the behavior can be across prompts. Table 4.8 therefore includes representative strong and weak cases from the headline benchmark.

Table 4.8: Representative per-prompt examples

Prompt	Plaintext continuation	FHE continuation	Similarity	Token acc.	BLEU	Interpretation
Machine learning is	`a powerful tool for analyz`	`a powerful tool for analyz`	1.0000	1.0000	1.0000	Perfect agreement on a strongly patterned continuation
Once upon a time	`, there was a young`	`, there was a young`	1.0000	1.0000	1.0000	Strong generative prior is preserved
The function returns	`the sum of all even`	`the sum of all numbers`	0.6852	0.8333	0.6687	Good semantic retention despite token-level drift
The capital of France is	`Paris.`	`the country's parliament`	0.1702	0.1667	0.0000	Factual recall degrades badly
Quantum computing uses qubits to	`perform computations. A`	`represent the state of a`	0.2573	0.1667	0.2403	Partially relevant but divergent continuation
In conclusion, the evidence suggests	`that the use of social`	`that the virus is a`	0.2296	0.5000	0.3021	Surface overlap survives more than semantic precision

These examples suggest that the encrypted system preserves **high-confidence language continuation** better than **specific factual retrieval**. The prompt set is small, so this pattern should not be overgeneralized, but it is consistent with the broader quantitative results.

4.10 Latency Analysis

Table 4.9: Latency and communication breakdown

Component	Average value
Total FHE time	3396.285 s
Server compute	2760.8911 s
Client encrypt	416.4706 s
Client decrypt	177.6092 s
Network	40.6634 s
ONNX runtime	1.7906 s
Upload size	81.7846 MB
Download size	19180.1842 MB
Total communication	19261.9687 MB
Projected server compute, 8-core (16 threads)	345.1114 s
Projected server compute, 48-core (96 threads)	57.5186 s

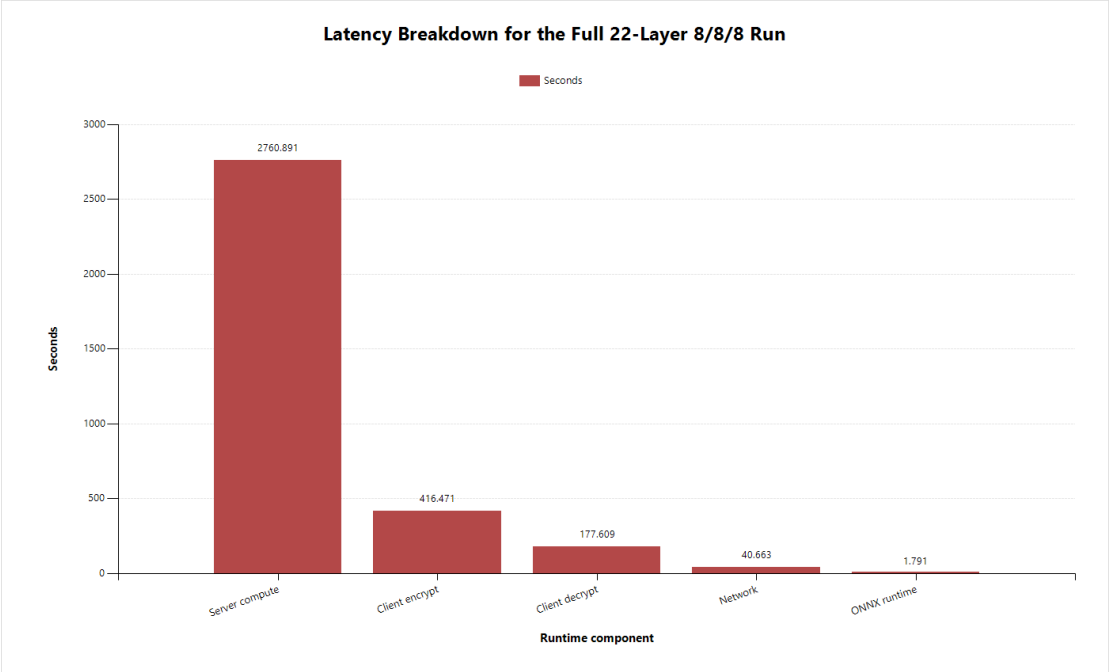

Figure 4.6: Latency breakdown of the headline benchmark.

The local testbed latency is dominated by encrypted server computation:

1. Server compute accounts for approximately **81.29%** of total runtime.
2. Client encryption and decryption together account for approximately **17.49%**.
3. Network transfer is only about **1.20%** of wall-clock time in the local testbed.

That last observation requires care. It would be a mistake to conclude that communication is unimportant. In **time**, network transfer is small in this local setup. In **volume**, communication is enormous, especially on the download side.

4.11 Communication Analysis and Compression Bottleneck

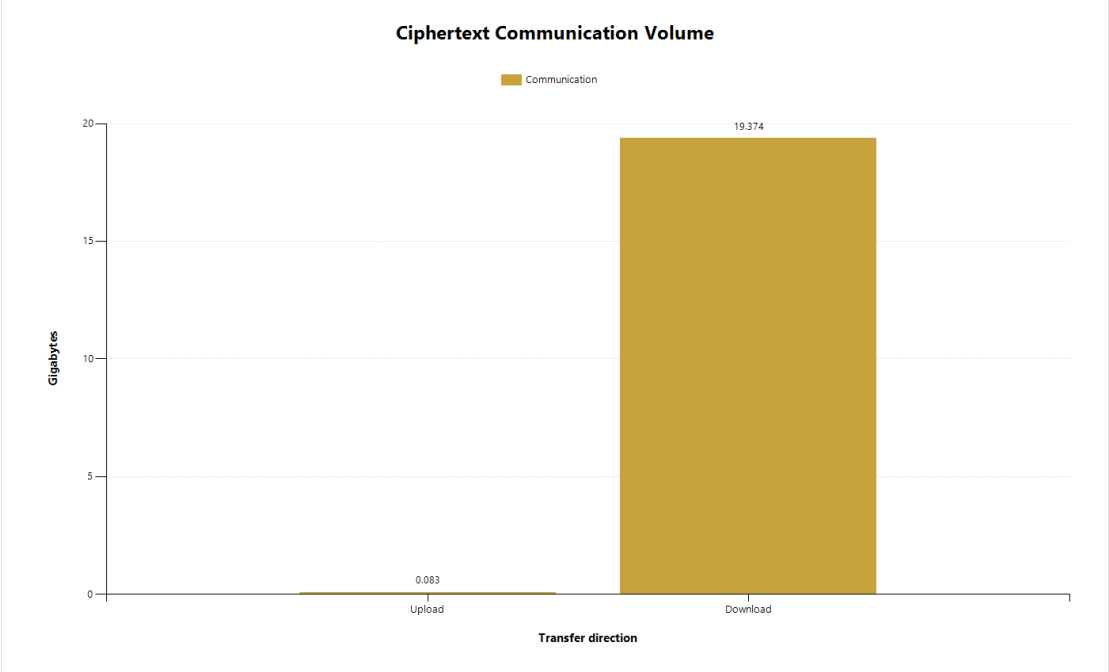

Figure 4.7: Communication volume by direction.

The benchmark uploads about **81.8 MB**, but downloads about **19.18 GB**, meaning roughly **99.58%** of the transferred bytes come from returned ciphertexts.

This communication pattern is one of the most consequential practical results of the study:

1. The current local benchmark is not network-time dominated because the client and server are effectively co-located for testing.
2. The system is nevertheless **communication-volume heavy**, which would matter much more in a real remote deployment.
3. The largest missing engineering lever is not generic transport compression but **practical output-ciphertext compression support** in the deployment path used by the system.

The experimental record shows that input/evaluation-key compression was enabled where available, and that naive gRPC gzip was not effective on ciphertext payloads. A matched two-layer comparison illustrates the point. Two March 2 non-gzip runs and two March 7 gzip-enabled runs produced the same average similarity (`0.6497`) and the same average logged payload volume (`1574.56 MB`), but average FHE time worsened from `270.984 s` to `414.725 s`, average server compute rose from `209.089 s` to `262.499 s`, and average network time rose from `5.333 s` to `75.777 s`.

The communication totals in this comparison come from application-level benchmark accounting rather than from packet-captured wire traces. Within that measurement framework, the result is clear: **naive gzip did not reduce the recorded ciphertext payload burden and it worsened runtime**. A future release that exposes output compression more directly at the

deployment level could therefore improve feasibility without changing the model split itself.

4.12 Idealized Scaling Projection

A simple proportional scaling model can be applied to the observed server-compute component by extrapolating from the measured two-thread baseline used in the benchmark runs to an 8-core/16-thread target and a 48-core/96-thread target.

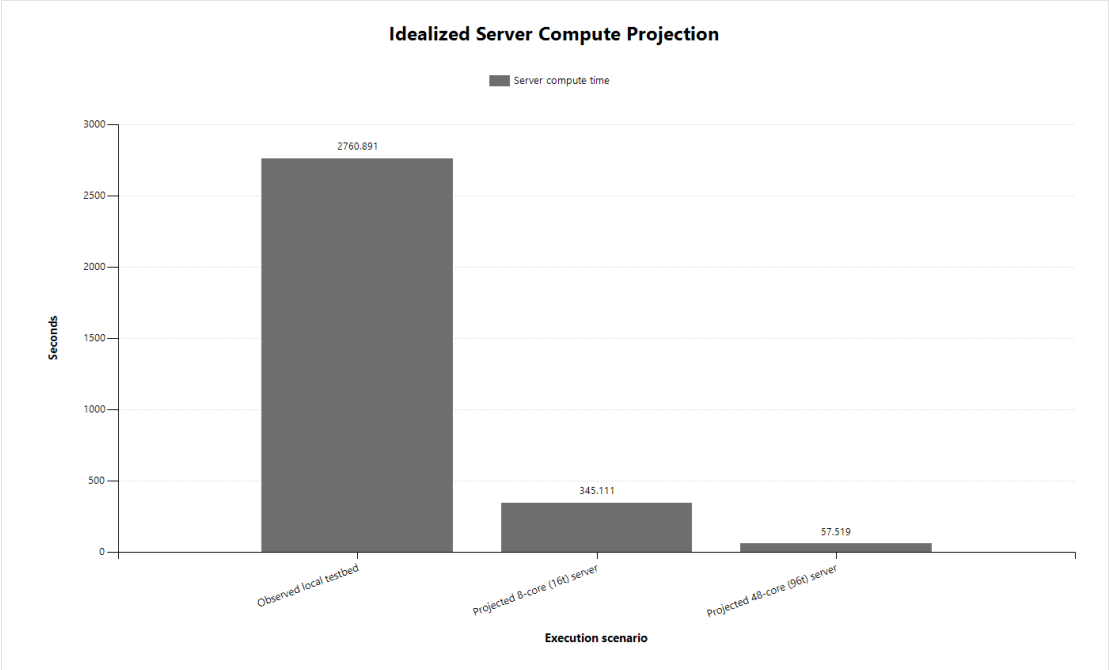

Figure 4.8: Idealized server-compute projection.

These numbers should be interpreted cautiously:

1. They are **projections**, not measured remote-server runs.
2. They assume near-linear improvement in the encrypted compute component.
3. They do not automatically include remote-network penalties.

Even with those caveats, the direction is informative. If the encrypted server component were moved from the observed two-thread environment to a more capable multi-core server, the system could become substantially more practical without changing the client logic.

That projection also sits within a broader technology trend. Official Zama material now documents GPU execution in TFHE-rs and hardware-accelerated backends such as HPU/FPGA, while its broader public roadmap extends toward dedicated accelerators [20, 21, 22]. These projections do **not** imply measured real-time performance for the present Concrete-ML deployment path. They do support the expectation that once comparable acceleration reaches the stack used here, server latency should fall substantially. At that point, the main architectural advantage of the present split would remain the same: it avoids spending encrypted compute on the transformer's non-linear bottlenecks in the first place.

4.13 Arithmetic Offload Potential of the Split

The main architectural argument of this project is not simply that the server can assist with inference. It is that the server can absorb the **linear bulk** of transformer computation while the client retains the smaller set of operations that are still comparatively inefficient or awkward under the present FHE stack.

Using TinyLlama's actual dimensions (`hidden_size = 2048`, `intermediate_size = 5632`, 32 query heads, 4 key/value heads), the affine work per transformer layer is:

1. `q`: `2048 x 2048 = 4,194,304`
2. `k`: `2048 x 256 = 524,288`
3. `v`: `2048 x 256 = 524,288`
4. `o`: `2048 x 2048 = 4,194,304`
5. `gate`: `2048 x 5632 = 11,534,336`
6. `up`: `2048 x 5632 = 11,534,336`
7. `down`: `5632 x 2048 = 11,534,336`

That gives **44,040,192 multiply-accumulate operations per layer** that are naturally server-offloadable if the server covers the attention projections and MLP linears.

In the delivered architecture, only the attention projections `q/k/v/o` are offloaded. Those four maps account for `9,437,184` multiply-accumulate operations per layer. On that basis:

1. The current system already offloads about **21.3548%** of transformer-body arithmetic to the server.
2. If the final `lm_head` is included in the accounting, the current end-to-end offloaded share is about **20.0063%**.

For comparison, a conservative client-side estimate at a representative `16`-token context, including RMSNorm, RoPE, local attention score/value products, SoftMax, SwiGLU activation, and residual adds, is about **152,064 scalar operations per layer**. If the next design step also moves the MLP linears `gate/up/down` to the server, the arithmetic balance changes dramatically:

1. Offloading `q/k/v/o + gate/up/down` moves about **99.6559%** of the transformer-body arithmetic away from the client.
2. If the final linear `lm_head` is also moved server-side, the end-to-end offloaded share rises to about **99.6776%**.
3. If `lm_head` remains local, the end-to-end offloaded share is still about **93.3625%**.

At the same representative context, the retained local non-linear path accounts for only about **0.3441%** of transformer-body arithmetic when measured as raw scalar operation count. That small numeric share is precisely the point: the non-linear path is not large in arithmetic volume, but under today's FHE toolchains it is disproportionately expensive because it relies on the least FHE-friendly operations in the transformer. The split therefore saves much more than arithmetic alone would suggest. It avoids spending encrypted budget on the part of the model with the worst

cryptographic cost profile.

The >99% figure refers to the immediate architectural extension in which the MLP linears, and optionally the final `lm_head`, are also moved server-side. It describes that next linear-extension scenario rather than the attention-only system delivered in this study. Even so, the current implementation already establishes the structural fact on which that extension rests: most transformer arithmetic is linear, while the local remainder is small in raw operation count but disproportionately expensive to reproduce under FHE.

4.14 Security Asymmetry and Deployment Positioning

From a deployment perspective, the architecture creates an explicit asymmetry. It strengthens confidentiality for the user's prompt against the server, but weakens model-provider secrecy relative to a pure server-side API. That trade-off defines the deployment niche for which the system is best suited.

For the user, the benefit is straightforward. A remote provider can execute the encrypted linear subgraph without seeing plaintext prompts or plaintext hidden states. This is useful when the user wants managed inference but does not want to disclose sensitive text to the provider.

For the provider, the picture is mixed. The current split allows the provider to keep operating a managed encrypted service, update server-side circuits centrally, and avoid distributing the full model as a normal local-inference package. However, the client still sees decrypted intermediates and local model packages, so this is not the right architecture if the primary goal is strong protection of proprietary model parameters [23], [24].

Compared with simply running the model locally, the hybrid system is most useful when:

1. the user cannot or does not want to host the full model stack locally;
2. the provider still wants to run a managed service;
3. prompt confidentiality from the provider matters more than strong provider-side model secrecy.

If a user can already run the full model locally with acceptable hardware cost and if distributing the model weights is acceptable, then local inference is often simpler and stronger for prompt privacy. The contribution of this project is not to replace all local inference, but to explore a middle point between local execution and plaintext cloud APIs. The architecture is best matched to privacy-sensitive managed inference, not to every possible inference setting. It is well aligned to deployments where prompt confidentiality from the provider is the main goal, and poorly aligned to deployments whose central business requirement is strict provider-side protection of model internals.

4.15 Hybrid FHE Versus Full-FHE

Prior work on encrypted transformer inference repeatedly highlights the difficulty of bringing SoftMax, normalization, and other non-polynomial operations into a practical FHE pipeline [6], [9]. Current Zama documentation makes that distinction even more explicit: linear arithmetic is the natural path, while non-linear functions are realized through table lookups or programmable-bootstrapping-heavy flows whose cost depends strongly on bit-width [17], [18]. That distinction explains why the present split matters.

Relative to a hypothetical full-FHE autoregressive stack, the hybrid design used here offers three practical advantages.

1. It avoids forcing SoftMax, masking, RMSNorm, SwiGLU, and KV-cache updates into the encrypted circuit budget.
2. It reduces encrypted compute, energy use, and hardware pressure even if powerful GPUs, FPGA/HPU accelerators, or future ASICs become available.
3. It makes it possible to offload the overwhelming linear bulk of the transformer while keeping the non-linear bottlenecks on the side of the system where they are still cheaper and easier to control.

The trade-off is coverage. Full-FHE would keep a larger fraction of the inference trace encrypted end to end and reduce client-visible intermediate values, but at a much higher cryptographic and systems cost. In that sense, the hybrid architecture is valuable not because it is superior to full-FHE on every axis, but because it is materially more efficient once current FHE weaknesses are treated as an architectural design input rather than as a temporary inconvenience. Even if future acceleration makes fuller encrypted execution feasible, the case for not encrypting avoidable non-linear work does not disappear.

4.16 Discussion

Taken together, the experiments support six conclusions.

1. The main systems contribution is a full 22-layer hybrid autoregressive runtime, not merely an isolated encrypted projection benchmark. The evidence for that claim is the real prefill/decode loop, the maintained KV cache and position state, the repeated server re-entry, and the 440 encrypted projection calls required for a full-length five-token completion.
2. The central architectural gain comes from matching current FHE strengths and weaknesses: server-side encrypted linear algebra, client-side non-linear and decode-state execution.
3. Extending encrypted execution across all 22 layers is possible, but it exacts a real cost in quality, latency, and communication volume under the present deployment path.
4. Calibration is a first-order design variable. Both calibration-source choice and layer-aware cascaded calibration materially affect full-depth behavior, and the same-family

calibration-toggle comparison shows that this is not a cosmetic preprocessing choice.

5. The privacy gain realized here is asymmetric: it strengthens prompt confidentiality against the server while leaving provider-side model secrecy comparatively weak.
6. The strongest obstacles to practical deployment are concrete engineering bottlenecks, especially encrypted server compute, output-ciphertext transfer, and the still-expensive treatment of non-linear functions in FHE. Just as importantly, the results show that compiler-side knobs such as calibration source, cascaded calibration, precision policy, and circuit error budget are first-order systems variables rather than preprocessing footnotes.

Chapter 5

Impacts of the Project

5.1 Impact on Societal, Health, Safety, Legal, and Cultural Issues

The most immediate societal implication of this project is that it widens the design space for privacy-sensitive use of language models. If part of inference can be offloaded without revealing the prompt in plaintext to the service operator, then managed LLM assistance becomes easier to justify in settings where the input itself is the sensitive asset. That includes legal drafting, internal policy work, financial analysis, institutional documentation, and other contexts in which the user may want model capability without surrendering raw text to an external platform.

What this project adds to that discussion is a concrete deployment pattern rather than a generic privacy claim. The report shows that the strongest practical lever is not ”encrypt everything at any cost,” but ”encrypt the transformer’s linear bulk while keeping the least FHE-friendly non-linear path local.” In this system, that selectivity is enforced not only by runtime placement but also by compile-time graph partitioning, calibration design, and parameter choice. That distinction matters for organizations that want a real privacy gain without committing immediately to a much heavier full-FHE stack.

This benefit should not be overstated. The system does not eliminate trust; it relocates it. The client remains trusted, some metadata remains visible, and provider-side model secrecy becomes weaker than in a conventional server-only API. For that reason, the architecture is better described as a **privacy-enhancing deployment pattern** than as a complete security solution.

From a legal and organizational perspective, that distinction matters. A privacy-enhancing design can strengthen internal governance and reduce unnecessary exposure, but it does not by itself establish compliance with any specific regulatory regime. Institutions would still need to consider data retention, endpoint security, auditability, and the handling of locally decrypted information. The contribution here is narrower and still worthwhile: reducing plaintext prompt exposure at the service boundary.

There is also a cultural dimension. One reason many users remain skeptical of AI systems

is not the capability itself, but the opacity surrounding who sees the data and how it is handled. Architectures that visibly reduce server-side visibility may therefore improve trust in privacy-sensitive AI workflows. At the same time, the project also highlights a countervailing concern: a deployment that favors user confidentiality may be less attractive to providers who rely on strict control of proprietary models. That tension is one of the real trade-offs brought into view by the architecture.

5.2 Impact on Environment and Sustainability

The environmental picture is mixed. The real sustainability value of the hybrid split is not that it avoids advanced hardware. It is that it avoids **unnecessary encrypted non-linear computation**. Even if future GPU, FPGA/HPU, or ASIC-backed FHE execution becomes commonplace, forcing every non-linear stage of a transformer into FHE would still consume more hardware time and energy than a split that leaves those functions in plaintext on the client.

That does not make the current system environmentally efficient. FHE remains computationally intensive, long benchmark runs consume significant server time, and large ciphertext transfers create additional storage and transmission overhead. What the present architecture does is reduce the scale of the encrypted problem to the part of the transformer where the offloading benefit is highest and the FHE penalty is most justified.

The sustainability lesson is therefore restrained but important. Selective encryption remains valuable not because future hardware will fail to improve full-FHE performance, but because avoiding unnecessary encrypted work is still the more energy-conscious and cost-conscious design even under stronger hardware assumptions. In that sense, the hybrid architecture is not only a response to current toolchain limits; it is also a statement about where encrypted effort is most productively spent.

Chapter 6

Project Planning and Budget

6.1 Project Timeline

Project execution followed iterative phases rather than a strict linear sequence. Early literature review and feasibility analysis defined the hybrid split, while later compiler and runtime work exposed calibration, masking, decode-state, and communication issues that required redesign. The timeline below reflects that phased progression across scoping, implementation, experimentation, and documentation.

1. Literature review and problem scoping.
2. Transformer/FHE feasibility study and model selection.
3. Compilation-pipeline construction and ONNX export.
4. Server implementation with gRPC, TLS, and session management.
5. Client implementation with reference-free local execution, prefill/decode control, and benchmark logging.
6. Shallow parameter screening, calibration design, cascaded-calibration refinement, deep ablation experiments, and result analysis.
7. Report writing and final document preparation.

Three technical milestones shaped the later schedule. The first was the initial end-to-end prefill/decode path, which established that the hybrid split could run as a real causal loop rather than as isolated encrypted calls. The second was the calibration redesign after masking, KV-cache, and decode-state alignment issues had been corrected. The third was stabilization of the final 22-layer benchmark families used in Chapter 4.

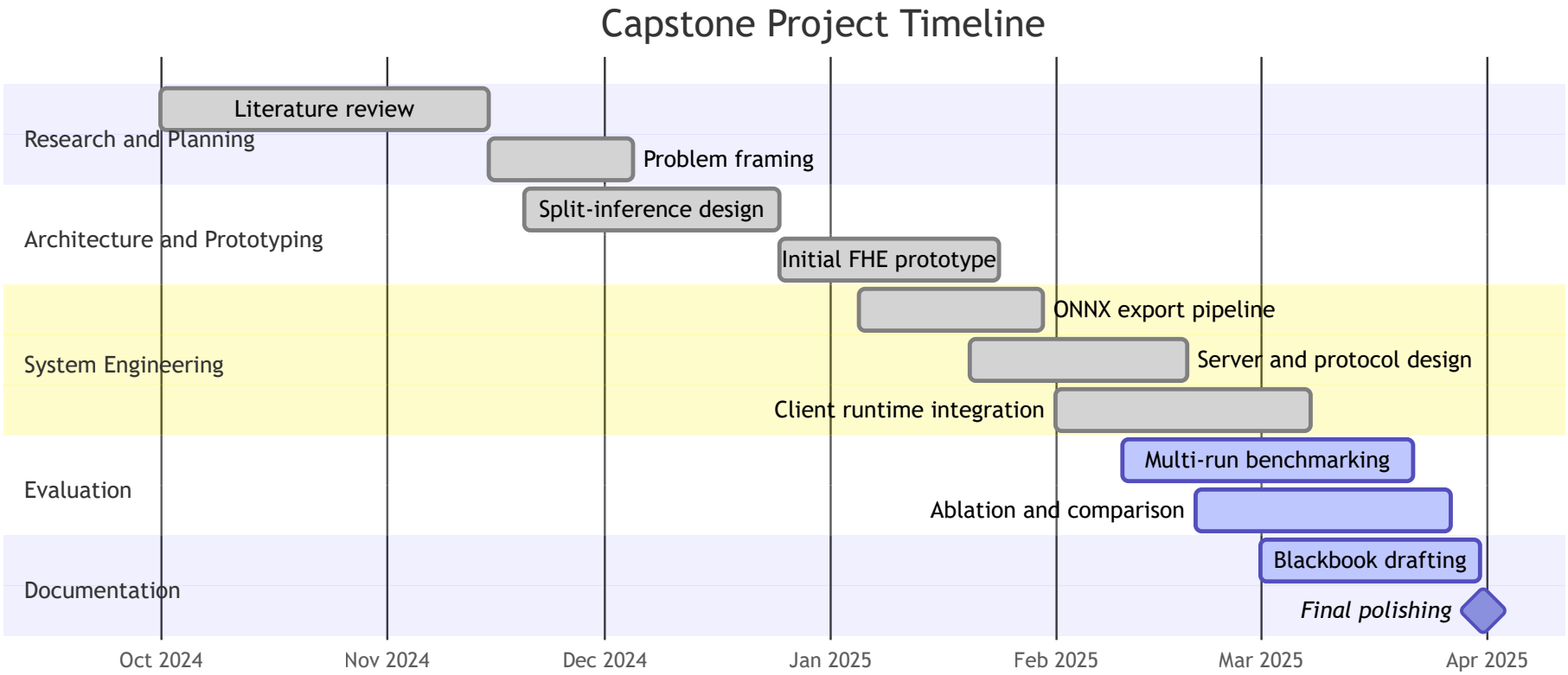

Figure 6.1: Capstone project Gantt chart

6.2 Budget

The project relied primarily on existing compute resources and open-source software, so the direct financial cost remained modest. The more significant burden came from long compilation and benchmarking runs, repeated failed experiments, and the engineering time required to stabilize a stateful hybrid FHE runtime together with its calibration pipeline.

Table 6.1: Budget and resource summary

Item	Description	Approximate cost (BDT)
Local validation system	Existing local compute environment; not purchased specifically for this project	0 incremental
Electricity and extended compute use	Long compile and benchmark sessions over the project period	1500
Internet connectivity	Repository access, package setup, documentation, and synchronization	2000
Backup and storage overhead	Local backups and removable storage used during the project	1500
Printing and binding	Final blackbook preparation	1500
Software licenses	Open-source toolchain used in this project	0
Total incremental estimate		**6500**

The main practical constraint was therefore not cash expenditure alone, but the amount of end-to-end validation that could be completed within the project period. That constraint shaped the decision to stop encrypted coverage at the attention projections, even though the broader architectural argument extends naturally to additional linear components.

Chapter 7

Complex Engineering Problems and Activities

7.1 Complex Engineering Problems (CEP)

This project qualifies as a Complex Engineering Problem because it sits at the intersection of several domains that do not naturally fit together: transformer inference, fully homomorphic encryption, distributed execution, quantization, and empirical systems evaluation. None of these areas is trivial on its own, and the central design question cannot be answered by optimizing only one of them. A decision that improves privacy may damage runtime. A choice that improves quality may increase encrypted depth or communication cost. A change that appears cryptographically sound may still fail as a usable system. These interacting constraints are what make the problem complex in the engineering sense.

Table 7.1: Complex Engineering Problem attributes in this project

CEP attribute	How it appears in this project
Depth of knowledge required	Requires understanding of transformer internals, FHE compilation, quantization, networking, and runtime systems
Conflicting requirements	Privacy, runtime, accuracy, memory use, and communication cost all conflict with one another
Analysis beyond routine practice	No off-the-shelf recipe exists for full 22-layer hybrid FHE TinyLlama deployment with real autoregressive decode, staged compiler calibration, artifact packaging, and this linear/non-linear split
Familiar and unfamiliar issues	Standard software engineering tasks are mixed with unfamiliar FHE deployment and ciphertext-communication issues
Significant consequences of decisions	Choosing what to encrypt, how to calibrate, how aggressively to set precision and `p_error`, and how to manage keys materially changes both security and model quality
Multi-domain constraints	Security, systems, performance, reproducibility, and user privacy all affect the design

7.2 Complex Engineering Activities (CEA)

The project also qualifies as a Complex Engineering Activity because it required more than straightforward implementation. The final system was shaped by repeated technical judgment: deciding the encryption boundary, keeping a fully autoregressive decode loop correct across that boundary, revising calibration when output drift became too severe, and redesigning the communication path when earlier transport choices proved wasteful. The work required system construction, instrumentation, measurement, and redesign in response to what those measurements revealed.

Table 7.2: Complex Engineering Activity attributes in this project

CEA attribute	Evidence from the project
Problem formulation	Reduced the broad problem of ”private LLM inference” to a feasible architecture that could be defended under capstone constraints, then expressed it as a coupled compiler-and-runtime system
System synthesis	Integrated encrypted server projections, reference-free local execution, staged compilation, autoregressive decode control, key handling, transport, and evaluation into one runtime
Investigation	Used ablations on layer count, bit-width, calibration source, cascaded calibration, `p_error`, and generation length to understand where the system succeeds and fails
Engineering judgment	Chose attention-only FHE for the delivered system, introduced layer-aware calibration and shallow parameter screening to stabilize depth, and deferred MLP-under-FHE because the broader split exceeded the validated resource and noise budget
Tool integration	Brought Concrete-ML, ONNX Runtime, PyTorch, gRPC, optional TLS and API-key controls, Prometheus, and artifact verification into one operational workflow
Validation and iteration	Treated debugging, instrumentation, parameter screening, packaging checks, and repeated reruns as part of the evidence-building process rather than as separate cleanup work

Chapter 8

Conclusions

8.1 Summary

This capstone set out to answer a practical question rather than a purely theoretical one: can a language model be split in a way that meaningfully improves prompt privacy while matching current FHE strengths and avoiding its most expensive non-linear bottlenecks? The system developed here shows that the answer is yes, but only within a narrow and carefully engineered operating region.

The project contributes in three connected areas. First, it delivers a runtime: all 22 TinyLlama layers contribute encrypted `q`, `k`, `v`, and `o` projections, yielding 176 deployable FHE circuits while the rest of the decode loop remains local. Second, it defines a method: an offline compilation pipeline that exports the reference-free local graph, constructs separate prefill and decode calibration corpora, uses real KV context for decode calibration, propagates calibration layer by layer through cascaded simulation, and calibrates `o` against quantized attention outputs rather than clean hidden states alone. Third, it documents the empirical consequences of that design through a focused study of quality, latency, calibration, communication, and deployment trade-offs across a full causal decoder.

The strongest operating point observed in the final evaluation uses full-depth uniform `8/8/8` quantization. That setting preserves enough output structure to remain analytically meaningful, but it also imposes heavy cost in runtime and communication. This judgment is grounded in a ten-prompt engineering benchmark with a maximum of five new tokens per prompt, so it should be interpreted as feasibility evidence rather than as broad deployment validation. Within that scope, the central conclusion is restrained but clear: the architecture is **credible as a privacy-preserving managed-inference baseline, but not yet ready for ordinary production use**.

8.2 Limitations

The principal limitations of the current system are:

1. **Latency.** Average end-to-end latency under the five-token-cap benchmark is approximately 3396 seconds.
2. **Communication volume.** Returned ciphertexts dominate transfer size, with more than 19 GB downloaded in the headline benchmark.
3. **Partial-model coverage.** The delivered system is hybrid, not fully homomorphic end to end.
4. **Provider-side model confidentiality.** The current split does not strongly protect proprietary model behavior or weights from a determined client.
5. **Quality degradation.** Output drift remains significant, especially for factual prompts and longer decode loops.
6. **Limited benchmark breadth.** The prompt set is small and intended for feasibility evaluation rather than broad product-level validation.
7. **Toolchain dependence.** Practicality is heavily affected by the current capabilities and limitations of the Concrete-ML deployment interface.

8.3 Future Work

The most important next steps are:

1. **Output-ciphertext compression support.** If the deployment path exposes practical output compression, the current communication bottleneck could be reduced substantially.
2. **More parallel and accelerated server hardware.** The measured compute split, together with the official GPU and hardware-acceleration direction in the broader Zama stack [20, 21, 22], suggests that stronger server environments could reduce runtime materially once those backends reach the deployment path used here.
3. **Extended linear coverage.** This project stopped at encrypted attention projections, but the same design logic naturally extends to the MLP linears and, potentially, the final linear tail. Under a conservative arithmetic-cost estimate, covering `q/k/v/o + gate/up/down` already offloads about `99.6559%` of transformer-body computation from the client, and adding the final `lm_head` raises the end-to-end offloaded share to about `99.6776%`.
4. **Improved quantization and calibration.** Quantization-aware fine-tuning, cleaner controlled A/B calibration studies, or stronger layer-aware calibration could improve quality retention.
5. **Stronger provider-side protections.** If commercial deployment requires stronger model secrecy, the split and trust assumptions would need to be revisited with additional protection mechanisms.
6. **Broader evaluation.** Larger prompt sets, varied generation lengths, and remote-server experiments would strengthen external validity.
7. **Alternative secure-computation hybrids.** Hybrid privacy mechanisms may offer better trade-offs for transformer non-linearities.

Taken together, these limitations and future directions clarify the contribution of the project. The study provides a concrete end-to-end case in which a deep linear-server / non-linear-client split is implemented as a real autoregressive TinyLlama runtime, the supporting offline compiler and calibration pipeline materially shape whether that runtime works at depth, longer rollouts expose the present stability limit, and calibration decisions remain first-order parts of the system rather than peripheral preprocessing choices. The retained `p_error = 0.1` default belongs in that same category: it was kept because a shallow screening pair showed essentially unchanged outputs and quality alongside modest runtime savings. With better ciphertext compression, stronger server hardware, broader encrypted linear coverage, and future acceleration backends, the same architectural direction could become substantially more useful than it is today.

Bibliography

[1] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On data banks and privacy homomorphisms," in *Foundations of Secure Computation*, 1978.

[2] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the 41st ACM Symposium on Theory of Computing*, 2009.

[3] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.

[4] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of the 33rd International Conference on Machine Learning*, 2016.

[5] R. Dathathri, O. Saarikivi, H. Chen, K. Laine, K. Lauter, S. Maleki, M. Musuvathi, and T. Mytkowicz, "CHET: An optimizing compiler for fully-homomorphic neural-network inferencing," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019.

[6] T. Chen, H. Bao, S. Huang, L. Dong, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei, "THE-X: Privacy-preserving transformer inference with homomorphic encryption," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.

[7] R. Mittal, "Improving inference privacy for large language models using fully homomorphic encryption," University of California, Berkeley, Tech. Rep. UCB/EECS-2024-225, 2024.

[8] Zama, "Concrete ml production deployment guide," 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/concrete-ml/guides/client_server

[9] I. Zimerman *et al.*, "Power-softmax: Towards secure llm inference over encrypted data," *arXiv preprint arXiv:2410.09457*, 2024.

[10] D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, "Encryptedllm: Privacy-preserving large language model inference via gpu-accelerated fully homomorphic encryption," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 12 677–12 688.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[12] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[13] P. Zhang, G. Zeng, T. Wang, and W. Lu, "Tinyllama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.

[14] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, 2017.

[15] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, 2012, pp. 309–325.

[16] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," Cryptology ePrint Archive, Paper 2012/144, 2012. [Online]. Available: https://eprint.iacr.org/2012/144

[17] Zama, "Table lookups basics," 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/concrete/2.9/operations/table_lookups

[18] ——, "Fhe op-graph design," 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/concrete-ml/explanations/inner-workings/fhe-op-graphs

[19] ——, "Inference," Concrete ML LLM documentation, 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/concrete-ml/llms

[20] ——, "Gpu acceleration," TFHE-rs documentation, 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/tfhe-rs/0.6-3/guides/run_on_gpu

[21] P. Gardrat, "Announcing hpu on fpga: The first open-source hardware accelerator for fhe," Zama blog, 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://www.zama.ai/post/announcing-hpu-on-fpga-the-first-open-source-hardware-accelerator-for-fhe

[22] Zama, "Zama confidential blockchain protocol litepaper," 2025, [Online; accessed 1 April 2025]. [Online]. Available: https://docs.zama.ai/protocol/zama-protocol-litepaper

[23] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction apis," in *25th USENIX Security Symposium*, 2016.

[24] R. Sun, Z. Sun, L. Lu, and A. Mislove, “Mind your weight(s): A large-scale study on insufficient machine learning model protection in mobile apps,” in *30th USENIX Security Symposium*, 2021.

Appendix A

Prompt Set Used in the Headline Benchmark

1. The capital of France is
2. The speed of light is approximately
3. Machine learning is
4. Quantum computing uses qubits to
5. Once upon a time
6. The detective examined the mysterious
7. To implement a binary search
8. The function returns
9. Hello! How can I help
10. In conclusion, the evidence suggests

Appendix B

Benchmark Protocol Summary

The main benchmark reported in Chapter 4 follows a fixed protocol so that comparisons remain consistent across runs.

1. **Model setting.** TinyLlama 1.1B is evaluated under the hybrid split in which only the `q`, `k`, `v`, and `o` projections are executed under FHE.
2. **Depth setting.** The headline experiment encrypts all 22 transformer layers.
3. **Precision setting.** The principal benchmark uses uniform `8/8/8` quantization.
4. **Circuit error budget.** The deeper runs keep `p_error = 0.1` as the default because the available two-layer screening pair showed identical prompt outputs and rounded quality metrics relative to `p_error = 0.01`, with modest runtime and payload improvements.
5. **Prompt set.** Ten short prompts are used, covering factual completion, coding-style prompts, narrative continuation, and assistant-style phrasing.
6. **Generation length.** The main benchmark uses five generated tokens per prompt. A separate 20-token run is reported only as a stress test for autoregressive error accumulation.
7. **Reference condition.** Each hybrid output is compared against a plaintext output from the same model family.
8. **Reported values.** Chapter 4 reports prompt-level averages unless stated otherwise.

The main comparative metrics are interpreted as follows:

1. **Token accuracy** is the fraction of generated tokens that match the plaintext reference at the same positions.
2. **First-token accuracy** records whether the first generated token matches the plaintext reference, then averages that result across prompts.
3. **BLEU** measures surface-form overlap between plaintext and hybrid outputs.
4. **Semantic similarity** measures how close the two outputs remain in embedding space even when exact token sequences differ.
5. **PPL ratio** is computed as `FHE_PPL / Regular_PPL`, so values above `1.0` indicate degradation relative to plaintext generation.
6. **Communication total** combines upload and download volume, while the communication analysis separately highlights that download dominates the total.

Appendix C

Arithmetic Offload Estimate

The offload percentages in Section 4.13 follow from TinyLlama's published dimensions and the projection sizes used in this implementation.

Per transformer layer:

1. Attention projections contribute

$$q+k+v+o = (2048\times2048)+(2048\times256)+(2048\times256)+(2048\times2048) = 9{,}437{,}184$$

2. MLP linears contribute

$$gate + up + down = (2048 \times 5632) + (2048 \times 5632) + (5632 \times 2048) = 34{,}603{,}008$$

3. Total linearly offloadable work per layer is therefore `44,040,192` multiply-accumulate operations.

For the client-retained side, Section 4.13 uses a conservative `16`-token-context estimate including:

1. two RMSNorm passes,
2. RoPE on `q` and `k`,
3. local `QKT` and `AV` attention products,
4. SoftMax,
5. local SwiGLU activation and gating,
6. residual additions.

Under that estimate the retained client work is about `152,064` scalar operations per layer, yielding:

1. **Transformer body offloaded:** `99.6559%`
2. **End to end offloaded if `lm_head` also moves server-side:** `99.6776%`
3. **End to end offloaded if `lm_head` remains local:** `93.3625%`

These are arithmetic-cost estimates, not measured runtime shares. They are included only to show that most transformer compute remains linearly offloadable.

Appendix D

Curated Calibration Prompt Corpus

The custom-calibration experiments in Chapter 4 did not reuse the ten benchmark prompts from Appendix A. They used a separate curated corpus of 256 short prompt prefixes designed to expose the compiler to a broader range of token patterns and hidden-state distributions before circuit generation.

The curated corpus was intentionally mixed across several domains:

1. factual and science continuations,
2. technology and machine-learning statements,
3. code prefixes and programming-language syntax,
4. narrative and creative-writing openings,
5. history, economics, and social-science prompts,
6. mathematics and logical statements,
7. assistant-style and instructional phrasing,
8. health, environment, law, arts, and sports examples.

Representative entries from that calibration corpus included:

1. `The largest ocean in the world is`
2. `Photosynthesis is the process by which`
3. `Climate change is caused by increased`
4. `Autonomous vehicles use sensors to navigate`
5. `def fibonacci(n):`
6. `SELECT * FROM users WHERE`
7. `The dark clouds gathered ominously as`
8. `The old lighthouse keeper watched as`
9. `The Roman Empire fell because of`
10. `Supply and demand determine prices in`

This corpus served only as calibration data. It was not used as the headline benchmark prompt set. Its role was to improve distribution matching for the compiled circuits relative to a generic text-only calibration source, which is why Chapter 4 compares it against a WikiText-2-derived alternative rather than treating it as an evaluation dataset.